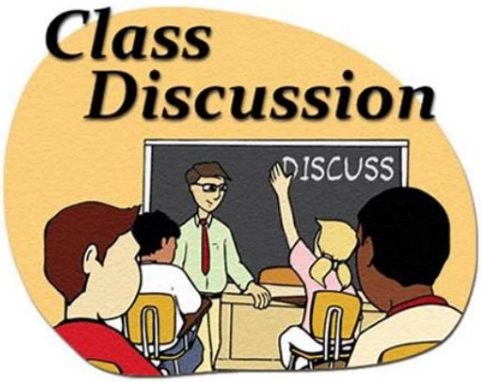
第五章 软件设计

架构是约束 AI 代码生成、控制系统复杂性、保证质量属性、支持团队协作和支撑系统演化的关键手段。

课堂讨论：为什么要进行设计





课堂讨论：建造一栋摩天大楼为什么要在施工之前进行设计？设计方案要考虑哪些方面的内容？如果是制作一个小板凳呢？



软件设计

- 软件设计是软件需求与实现代码之间的桥梁，起着承上启下的作用
  - ✓ 承上：为需求实现提供了一种抽象的解决方案规划，明确了功能如何分配到不同的软件单元以及非功能性的质量需求（例如可扩展性、性能、可靠性等）以何种方式实现
  - ✓ 启下：明确了软件单元的开发要求（例如接口定义、功能要求等），使得它们可以以一种分而治之的方式分别实现
- 应对系统复杂性的需要
  - ✓ 随着软件规模增大，复杂性成为软件开发的一个主要挑战
  - ✓ 人类应对复杂性的基本手段是分解加抽象
  - ✓ 需要决定如何将软件分解为多个组成单元（例如组件、模块、文件或类）并考虑它们的交互关系和集成方案
- 应对系统演化性的需要
  - ✓ 如果没有相应的设计考虑，那么软件响应变化的成本可能很高
  - ✓ 需要通过设计对未来可能发生的变化做好准备，例如软件各部分之间保持相对独立和松耦合、通过可扩展接口支持新功能和新特性的引入

软件设计的目标

- 软件需求和实现代码之间的桥梁：通过对于软件解决方案的规划回答软件需求如何实现的问题，同时为各部分的编码实现提出明确的要求
- 应对软件开发的复杂性挑战：通过分解和抽象将待开发的软件分为一系列组成部分，开发人员每次只需要专注于某一个部分而只对其他部分的抽象特性（例如接口）进行了解
- 应对软件开发的变化性挑战：通过良好的分解和抽象使得软件的各个组成部分保持相对独立和松耦合，同时通过灵活配置和可扩展接口等手段为未来可能的需求变化做好准备

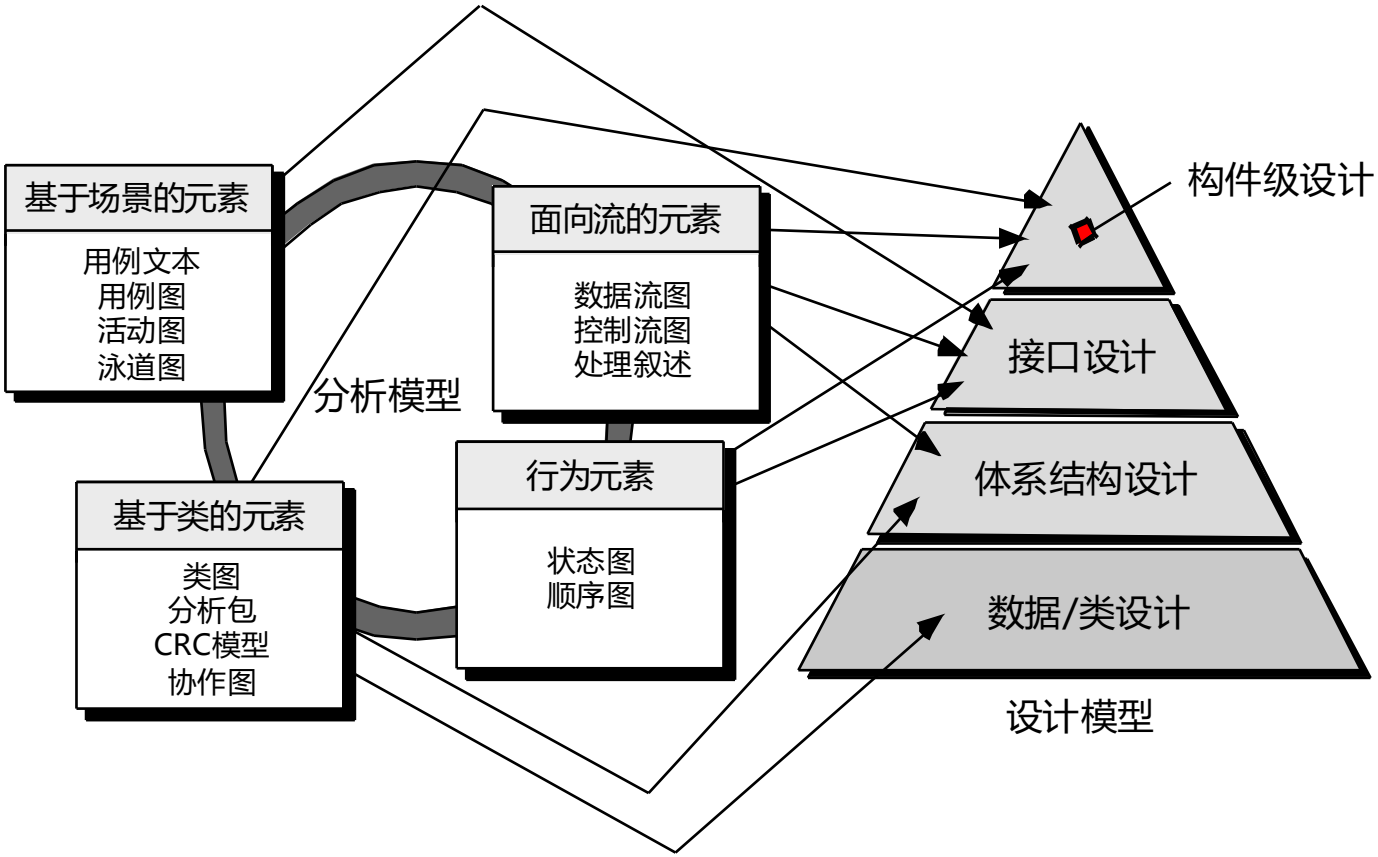
软件设计的层次

- **数据设计（类设计）**：软件的全局数据结构设计，包括数据实体及其属性和相互之间的关系
- **体系结构设计**：决定系统的高层分解结构以及各种全局性的设计决策（如开发语言、异常处理策略）
- **接口设计**：组件之间的交互接口设计，包括接口的功能和非功能性（如性能、吞吐量）要求、接口交互协议、接口操作及实现类定义等
- **构件（组件）级设计**：组件内的具体设计方案，如类及类之间关系、组件内部局部数据结构和算法设计等

AI智能体辅助软件工程中设计任务

任务
任务 1：提取架构关注点
任务 2：生成候选架构
任务 3：评价并选择架构
任务 4：确定技术栈
任务 5：设计模块与依赖
任务 6：设计接口契约
任务 7：设计可测试性

软件设计层次图



需求模型到设计模型的转换

软件设计思想

- 分解（Decomposition）与抽象（Abstraction）
- 软件体系结构（Software Architecture）
- 关注点分离（Separation of Concerns）
- 模块化（Modularity）
- 信息隐藏（Information Hiding）
- 重构（Refactoring）
- 复用（Reuse）

分解与抽象

- 分解和抽象是人类应对复杂性的两个基本手段，也是软件设计思维的重要基础
  - ✓ 分解：将软件不断地分解为更细粒度的代码单元，例如从组件、模块到文件和类
  - ✓ 抽象：忽略无关细节，只保留与当前问题相关的关键信息，例如接口抽象使得我们可以针对接口进行编程而忽略内部数据结构、算法等实现细节
- 分解往往需要与抽象的思想相结合才能发挥应对复杂性的作用
- 数据抽象和过程抽象
  - ✓ 数据抽象：对于目标对象的数据化抽象描述
  - ✓ 过程抽象：对一系列过程性步骤和指令序列的整体抽象

软件抽象的选择

- 好的抽象应该屏蔽底层细节，突出事物的本质特性，同时符合人的思维方式，从而实现降低复杂性的目标
- 由于针对抽象编程的实现方案不依赖于许多无关细节，因此好的抽象还能极大提高程序的可迁移性和可复用性
  - ✓ 例如，针对抽象的设备（例如抽象的打印机）编程的代码可以很容易地与不同型号的设备（例如具体的打印机）一起工作，只要这些设备都能实现同样的抽象设备接口（例如接受打印作业、返回打印机状态等）

软件体系结构

- 大规模软件系统（特别是网络化软件系统）需要更高层的设计考虑
  - ✓ 包含大量的代码，不同部分代码之间的交互关系复杂，经常还涉及跨进程、跨网络的通信
  - ✓ 往往有非常高的性能、可靠性、可维护性、可扩展性等非功能性质量要求
- 软件体系结构给出了软件系统的顶层设计
  - ✓ 包括一组软件组件、它们的外部属性以及相互之间的关系
  - ✓ 其他软件系统的全局的实现约定（例如编程语言、异常处理策略、数据库等资源使用方式）
- 充分体现了分解与抽象的基本原则
  - ✓ 给出了系统分解结构，使得开发人员可以负责不同部分
  - ✓ 给出了系统实现方案的抽象表示，开发人员可以在无需了解实现细节的情况下理解其他组件并考虑集成关系

体系结构中的分解、抽象、约定...

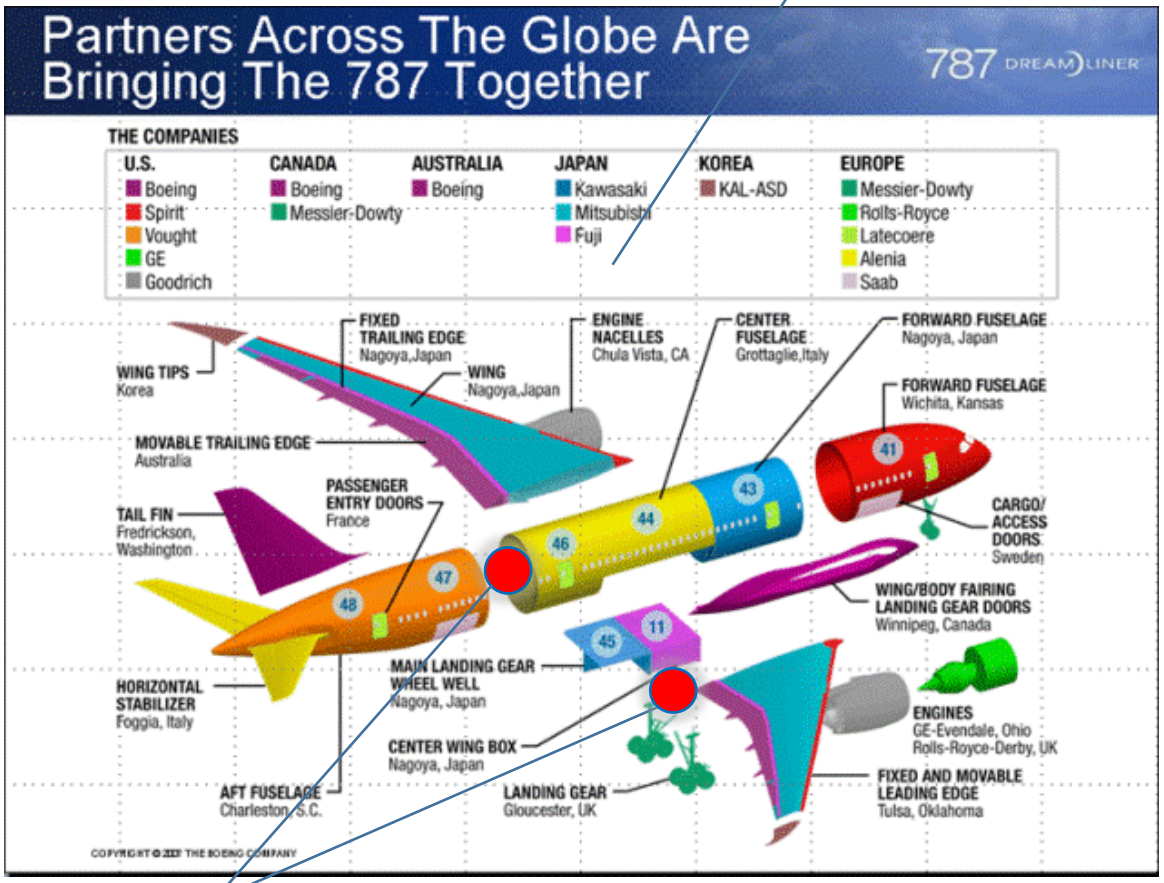
分解

约定

约束

标准

.....



接口（抽象）

总体质量在很大程度上取决于体系结构设计

软件体系结构的作用

- 软件项目分工合作的基础
  - ✓ 组件划分是不同开发小组和开发人员任务分配的重要依据
  - ✓ 软件组件的外部属性为每个组件的开发提出了具体要求
  - ✓ 组件间关系定义以及其他全局性的约定为这些组件的最终顺利集成提供了基础和保障
- 确认设计方案是否能有效满足需求
- 为满足未来演化和复用的需要而做出规划
- 作为多种候选技术方案对比选择的依据
- 识别并降低软件实现的风险
- 作为相关涉众沟通和交流的基础

关注点分离

- 关注点：软件系统中所实现的**某种功能**或**特性**，反映不同涉众所关注和关切的方面
  - ✓ 一卡通系统中的基本功能关注点：刷卡识别师生身份、通过手机支付进行充值、刷卡付款消费
  - ✓ 信息安全关注点：数据加密传输、所有操作都要进行日志记录
- 关注点分离
  - ✓ 将软件系统的整体需求分解为尽可能小的关注点并分解到不同的模块单元（例如模块、包、类、方法等）中实现
  - ✓ 不同的关注点尽量不要混杂在一起
- 模块化划分就体现了关注点分离的思想
  - ✓ **类作为一种基本的模块单元对关注点进行了划分**，与相同或相近的关注点相关的职责和功能被分配到同一个类中
  - ✓ 表示视图的用户界面与业务数据表示、处理数据的业务逻辑以及衔接用户界面与业务逻辑的控制逻辑相分离

横切关注点的问题

- 软件设计中的关注点散布和混杂问题
  - ✓ 散布：与同一关注点相关的职责散布在多个模块单元中
  - ✓ 混杂：同一模块单元中又混杂了多个关注点的职责
- 例如，在面向对象软件设计中
  - ✓ 很多类都需要执行身份认证、权限检查、日志记录等处理
  - ✓ 这些所谓的横切关注点很难被封装在单个类中，因此造成了相关职责的散布和混杂
- 面向切面的编程（简称AOP）为这种横切关注点的封装提供了一种有效的手段
  - ✓ 横切关注点可以被封装为切面（Aspect）
  - ✓ 切面与各个类中自身的主要关注点相分离
  - ✓ 切面通过声明式的方式编织到需要执行相关处理的地方

模块化

- 模块化：将整个产品或系统分解为大小合适、相对独立的模块
  - ✓ 使得各个模块的生产制造可以相对独立地进行
  - ✓ 可以在不破坏整体结构的基础上实现模块替换和扩展
  - ✓ 适用于软件设计的各个层次，包括软件体系结构中的组件、组件内的类一级的详细设计
- 分解和抽象设计思想的集中体现
  - ✓ 模块结构给出的系统分解方案可以以分而治之的方式实现
  - ✓ 模块接口为外部调用和整体集成提供外部抽象
- 模块化设计强调模块独立性
  - ✓ 一般可以用内聚度和耦合度来衡量
  - ✓ 好的模块化设计应实现模块的高内聚和低耦合



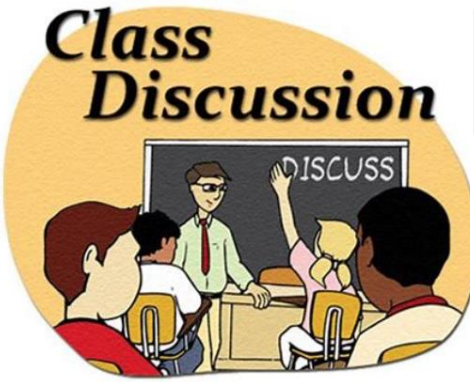
信息隐藏

- 模块（例如组件或类）将实现细节隐藏在内部，仅通过受限的接口对外提供访问
- 如果将模块内部的实现细节都对外暴露，那么即使模块分解得当也会造成不必要的模块间耦合
- 面向对象软件设计与实现中的信息隐藏
  - ✓ 一个包（package）中作为“门面”（facade）让外部可见的一些类的访问修饰符可以设置为public，而其他对外隐藏的类可以设置为protected
  - ✓ 一个类（class）中作为对外接口一部分的属性和方法（一般建议属性不要直接对外开放）可以设置为public，而其他对外隐藏属性和方法可以设置为private

信息隐藏的好处

- 通过屏蔽实现细节以及暴露抽象接口的方式降低了其他模块开发者对于当前模块的认知复杂性
- 通过抽象降低了内部实现细节的变化对于其他模块的影响
- 通过受控接口提供访问，可以更好地实现对于内部数据和操作的保护

课堂讨论：信息隐藏




```
public class CampusCard{
   //消费记录数组
   public Consumption [] consumpList;
}
```

```
public class CampusCard{
   //消费记录数组
   private Consumption [] consumpList;
   //获取消费记录条数
   public int getConsumpNum(){
      …
   }
   //按照指定序号返回一条复制的消费记录
   public Consumption getConsumptionAt(int index){
      …
   }
}
```

课堂讨论：校园一卡通系统中的校园卡类的两种设计方案哪一个更合理?

左边的设计方案会带来不必要的耦合：访问校园卡类的其他类将依赖于校园卡类中的消费记录数组，该内部数据结构发生变化，那么访问校园卡类的其他类也将不得不修改；外部其他模块有可能对数组中的消费记录内容进行修改

右边的设计方案避免了这种不必要的耦合，只要接口方法（public方法）不变那么其他类就无需修改

软件重构

- 在不改变代码外在行为的前提下，对代码作出修改以改进程序的内部结构（设计）
- 目的：提高软件的可维护性和可扩展性
- 软件经过不断修改后内部的设计和实现结构也会退化，重构可以阶段性地改进设计和实现质量，缓解内部质量退化的趋势
- 重构可以在多个层面上发生
  - ✓ 在保持软件系统整体行为不变的情况下对其体系结构的大规模重构
  - ✓ 在保持软件模块整体行为不变的情况下对其类级别上的详细设计进行设计重构
  - ✓ 在保持类行为不变的情况下对其内部设计进行代码级重构

敏捷开发中的软件重构

- 软件重构是敏捷方法所推崇的一种重要的开发实践
- 实现所谓的演化式设计（即随着迭代化的开发过程逐步完善设计）的一种重要手段
  - ✓ 强调要避免过度设计，即不要在软件开发初期基于对于需求的发展变化进行预测并在此基础上作出完整的设计方案
  - ✓ 推崇在增量和迭代化的特性实现过程中不断发现设计中的问题并通过重构不断进行改进
  - ✓ 应当持续对软件设计质量进行分析，及时发现各种设计问题及相应的改进机会，并通过重构实现软件设计改进

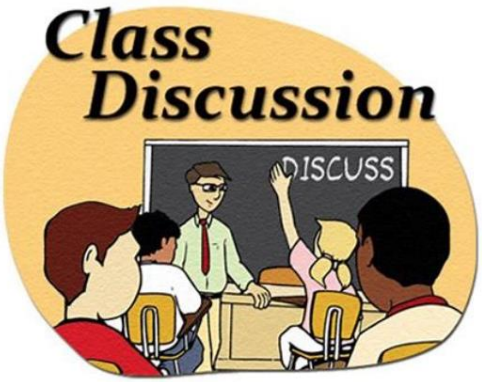
敏捷开发的观点：

好的设计是演化出来的而不是提前设计出来的

软件复用

- 在不同软件系统中或同一软件系统的不同部分重复使用相同或相似的软件代码、软件设计或其他相关的软件知识
- 不同层次的软件复用
  - ✓ 源代码复制粘贴（不推荐）
  - ✓ 软件库（包括编程语言开发库以及第三方库）API调用
  - ✓ 软件设计模式（需要结合具体问题需要进行实例化）
  - ✓ 面向特定类型软件应用的软件框架（例如Spring框架）
- 软件复用的两个方面
  - ✓ 面向复用的软件开发：为未来的软件开发提供软件复用机会和可复用软件资产，为此需要考虑共性抽象和封装
  - ✓ 基于复用的软件开发：利用潜在复用机会进行开发，需要考虑适合于当前项目的软件框架、设计模式和软件组件

课堂讨论：代码复制粘贴





课堂讨论：为什么企业软件开发过程中并不鼓励通过代码复制粘贴来实现软件复用？

面向对象软件设计

- 面向对象设计方法在组件级设计以及组件接口设计上扮演着重要的角色
- 面向对象软件设计内容
  - ✓ 通过面向对象设计方法来**确定所需要定义的类和接口及其之间的关系**
  - ✓ **通过UML图等方式描述面向对象设计方案**
  - ✓ 按照模块化设计的思想，面向对象设计中的类应当相对独立（通过内聚度和耦合度来衡量）
  - ✓ 面向对象设计存在一些常用的基本原则，可以为开发人员提供具体的设计指导
  - ✓ 对于横切关注点以切面（Aspect）的方式对其进行封装并与相关的类解耦

面向对象软件设计过程

- 面向对象软件设计的起点
  - ✓ 规模较大的软件系统：给定软件组件的设计要求，包括整体功能、对外接口等
  - ✓ 规模较小的软件系统：整个系统的软件需求（使用面向对象方法实现整个系统）
- 面向对象软件设计的基本步骤
  - ✓ 识别设计类
  - ✓ 明确设计类职责和协作
  - ✓ 细化设计类内部细节

潜在设计类的来源

- 来自问题域中的设计类
  - ✓ 来自现实世界问题域，往往在软件需求中进行刻画和描述
  - ✓ 包括名词性的业务实体对象（例如图书、读者）以及具有业务含义的处理过程（例如学生注册）
- 位于接口上的设计类
  - ✓ 待开发的软件系统与用户、硬件以及其他软件系统之间都有可能存在相应的接口（包括用户界面）
  - ✓ 包括实现软硬件接口包装以及用户界面自身的设计类
  - ✓ 也包括封装接口上所传递的信息（如请求参数）的设计类
- 与基础设施相关的设计类
  - ✓ 软件系统中的业务逻辑处理需要在计算、存储、网络等基础设施基础上才能实现
  - ✓ 例如实现数据库访问、跨网络或跨进程通信的设计类等

判断潜在设计类的合理性

- 是否包含多个同类对象所共有的且是当前软件系统所需要的属性及相关操作
- 例如，对于校园一卡通系统而言，刷卡消费点是一个重要的问题域实体，但是否将其作为设计类则需要考虑系统需要
  - ✓ 如果需要对刷卡消费点进行专门的管理，例如管理消费点的开设和关闭、统计各个消费点的消费额等，那么消费点实体具有多个系统需要属性和操作，可以作为一个设计类
  - ✓ 如果刷卡消费点不需要做专门的管理而只是在消费记录中简单进行备注，那么将消费点作为其他设计类（例如刷卡请求、消费记录等）的一个字符串类型的属性即可

明确设计类职责和协作-1

- 每个设计类所承担的职责的实现
  - ✓ 利用自身能力来完成：体现为所包含的属性（具有的信息和知识）以及方法（能够执行的操作）
  - ✓ 依赖于与其他类的协作来实现，包括类间继承、接口实现、功能依赖、属性访问等
- 职责与协作在很大程度上决定了设计质量
  - ✓ 实现类之间关注点分离和相对独立性，同时支持未来扩展
  - ✓ 综合考虑设计类的数量和粒度，避免职责过于集中或分散
- 按照职责单一、相对独立的原则确定类与类之间的职责分配和边界划分
  - ✓ 纵向的业务关注点分离，例如关注图书基本信息及其查询的图书类应当与关注出借状态和位置的图书副本类相分离
  - ✓ 横向的技术层次划分，例如属于用户界面层、业务层、数据层等不同层次的设计类相分离

明确设计类职责和协作-2

- 在关注点分离的基础上按照高内聚、低耦合的标准衡量每个类的独立性并进行调整
  - ✓ 将内聚度较低的类拆分成多个更小但更内聚的类
  - ✓ 将多个耦合度较高的类的部分职责进行合并
- 进一步考虑通过引入抽象类和接口等手段进行一些共性抽象，以进一步降低类间耦合同时更好地支持未来可能的扩展
  - ✓ 例如，将一个类对另一个类的具体依赖变成对一个抽象接口的依赖，这个抽象接口仅保留所依赖的操作的最小集合
- 类与类之间的协作关系：凡是通过自身能力无法完成的部分职责就需要依赖于协作

细化设计类内部细节

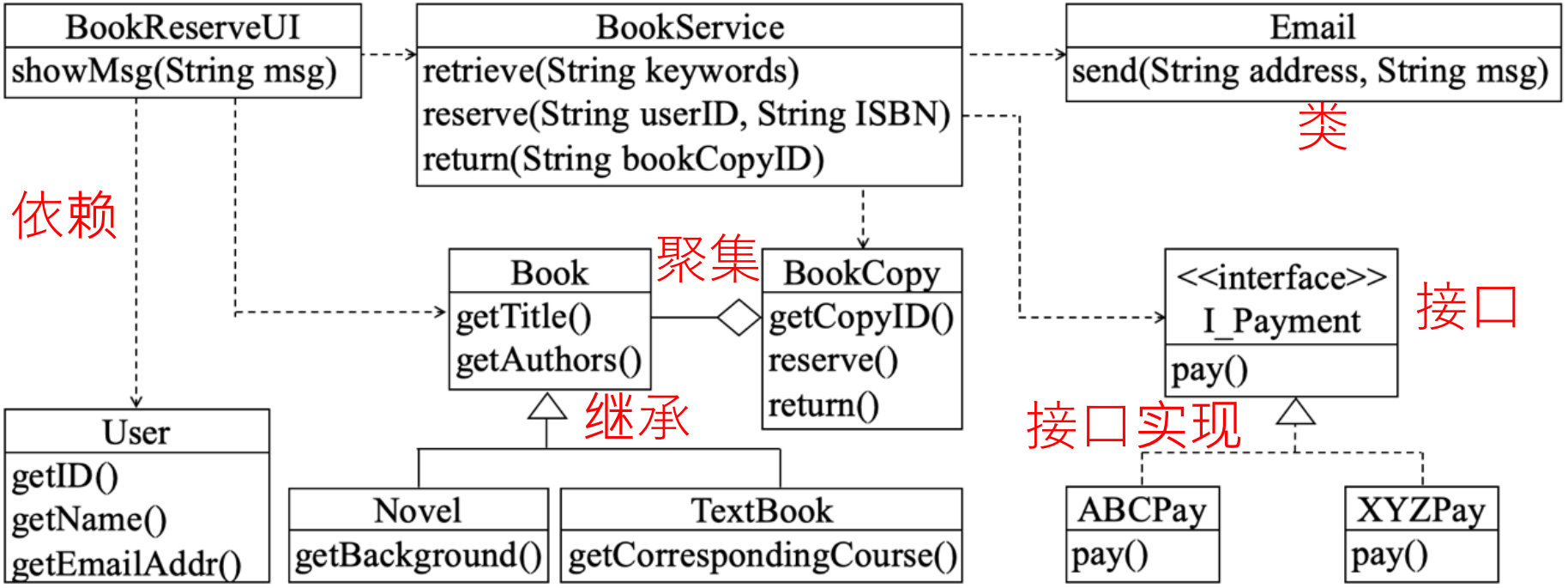
- 类属性需要细化的主要是访问修饰符、属性类型及初始值（缺省值）
  - ✓ 避免将属性设置为public，对于属性的访问（读取或修改）一般应当通过方法以一种受控的方式来实现
- 方法需要细化的主要是访问修饰符、参数及类型、返回值类型、前置及后置条件
  - ✓ 考虑将作为类的对外接口一部分的方法设置public
  - ✓ 方法的前置及后置条件是指方法执行之前和之后参数、返回值及类的状态变量（属性）等应当满足的条件，它们定义了方法及其调用者之间的契约
- 类内部需要用到的局部数据结构和算法
  - ✓ 算法一般可以通过流程图、伪代码等方式进行描述
  - ✓ 类内部使用的数据结构和算法一般应当对外隐藏

面向对象设计描述

- 面向对象软件设计的描述主要围绕设计类的职责和交互关系来进行
- UML（Unified Modeling Language）为这些描述提供了丰富的图形种类
- 常用的设计描述
  - ✓ UML类图描述面向对象的静态设计结构，包括类及类之间的静态关系
  - ✓ UML顺序图描述在特定场景中类与类之间的动态交互关系，例如类之间的消息发送及其先后顺序等
  - ✓ UML状态机图通过状态转换的方式描述类的实例对象的行为
- 其他设计描述
  - ✓ UML包图以包的形式描述更大粒度的模块之间的静态结构关系
  - ✓ UML通信图在展示类的实例对象间关系的基础上描述对象间消息发送的顺序
  - ✓ UML活动图描述由类方法调用构成的业务处理过程及算法流程

静态结构设计描述（UML类图）

校园一卡通系统中的图书馆管理子系统静态结构设计



类：表示为矩形组合，上下两部分分别是类名和方法（这里省去了类的属性、方法返回值和一部分方法）

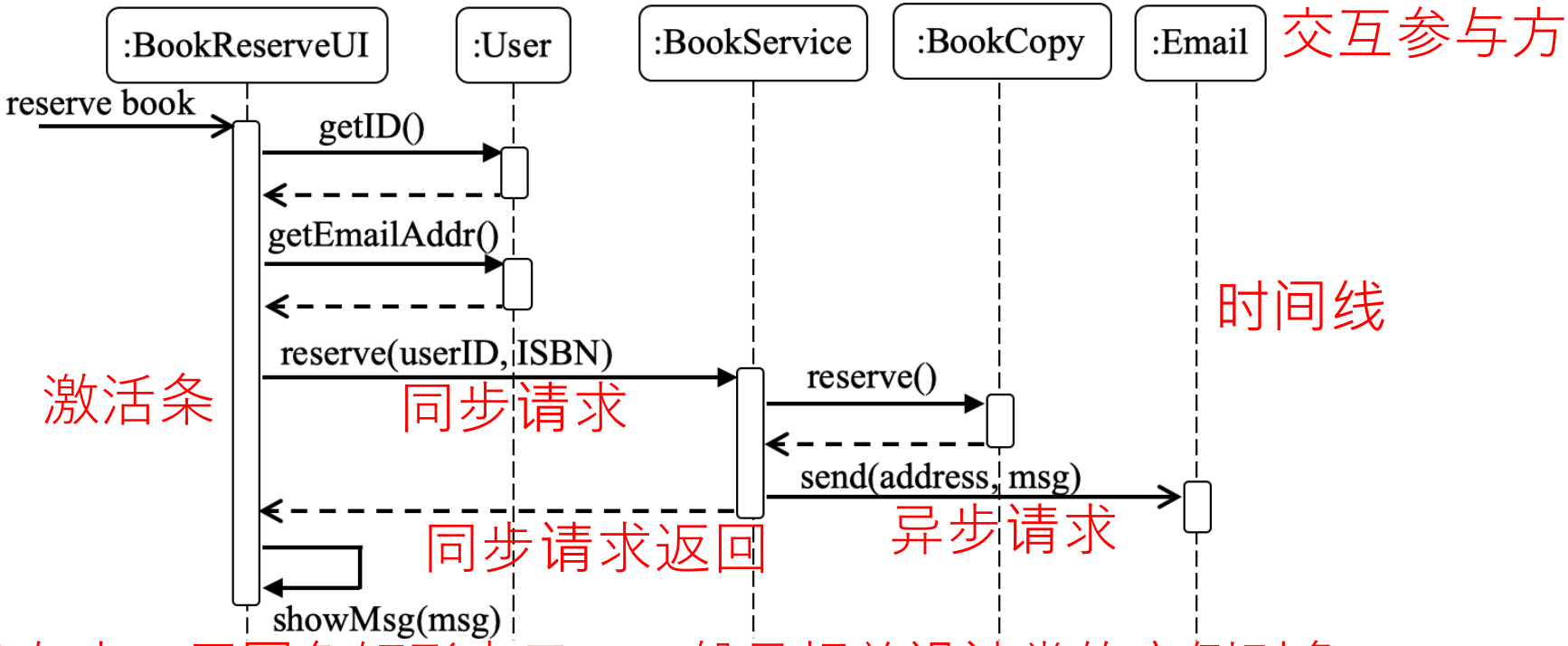
接口：利用UML的构造型（stereotype）机制定义的一种特殊的“类”

关系：箭头表示类与类之间的各种静态结构关系

- ✓ 继承：类之间的继承表示一种泛化和特化关系（isA）
- ✓ 聚集：类之间的聚集关系表示一种整体和部分关系
- ✓ 依赖：类之间的依赖关系表示方法调用、属性访问等功能依赖

动态交互关系描述（UML顺序图）

校园一卡通系统中的图书馆管理子系统动态交互设计



交互参与方：用圆角矩形表示，一般是相关设计类的实例对象

时间线：参与方下面的虚线，从上到下表示时间先后顺序

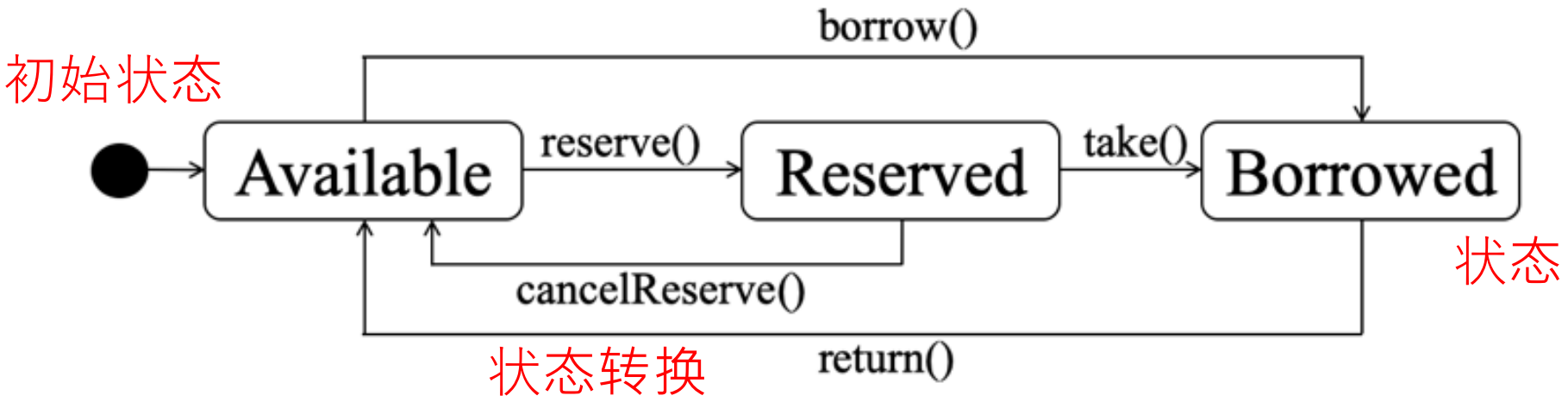
激活条：时间线上的矩形条，表示参与方在此期间处于激活状态

消息：不同参与方之间的横向箭头

- ✓ 同步请求消息：用实线三角形箭头表示（参与方发出请求消息后阻塞等待）
- ✓ 同步调用的返回消息：用虚线线型箭头表示
- ✓ 异步请求消息：用实线线型箭头表示（参与方发出请求消息后不进行阻塞等待）

行为设计（UML状态机图）

校园一卡通系统中的图书馆管理子系统的BookCopy类的行为设计



UML状态机图通过状态转换的方式描述类的实例对象的行为
状态：用圆角矩形表示，实心圆表示一个特殊的初始状态
状态转换：用箭头表示，箭头上的方法表示触发状态转换的方法调用

衡量模块独立性的内聚度和耦合度

- 模块独立性通常用内聚度和耦合度来衡量
- 高内聚的模块化设计
  - ✓ 强调模块的职责应该明确且专一，而且所包含的内容都是与该职责密切相关、不可或缺的部分
  - ✓ 常见的内聚形态：功能内聚、层次内聚、通信内聚、顺序内聚、过程内聚、时间内聚、功用内聚
- 低耦合的模块化设计
  - ✓ 模块间依赖尽可能少，理解和修改一个模块时不用过多考虑其他模块，修改后对其他部分的影响较小
  - ✓ 模块更容易被单独取出并在其他项目中进行复用
  - ✓ 常见的耦合形态：内容耦合、共用耦合、控制耦合、印记耦合、数据耦合、过程调用、类型使用、文件包含和包引入、外部依赖

常见的内聚形态

高

低

内聚形态	含义	适用层次	例子
功能内聚	模块仅执行单个计算任务并返回结果，并且不存在任何副作用（例如修改数据库或文件、产生界面提示等）。	操作（即方法或函数）	一个计算数学函数（例如sine、cosine）的方法
层次内聚	模块中仅包含一组密切相关的功能或服务，这些功能或服务形成一个严格的层次结构，其中高层可以访问较低层次上的功能或服务，反之则不行。	包、类	一个类对外提供生成成绩单的方法，内部的学生信息验证、成绩信息读取等方法构成严格的调用层次
通信内聚	模块中仅包含访问同样一组数据的操作。	类	一个购物车类中所包含的都是针对购物车数据的操作，例如添加商品、移除商品、修改商品数量等。
顺序内聚	模块中包含一组顺序性的处理过程，每一个过程都为后一个过程提供输入。	包、类	一个文本识别组件（包）包含三个顺序执行并存在输入输出关系的过程：位图区域划分、区域内字符识别、整体文本识别。
过程内聚	模块中包含一组依次执行的处理过程，但过程之间并不存在输入输出关系。	包、类	一个实现报表打印的类由分别打印表头、表格内容、表尾的方法组成
时间内聚	模块中包含一组在软件运行的某个阶段一起被执行的处理过程，但过程之间没有明确的顺序关系	包、类	一个负责系统初始化的类由一组分别执行不同初始化功能的方法组成
功用内聚	模块中包含一组主题相关的功能实现，这些功能可以单独被调用。	包、类	一个数学函数类（如java.lang.Math）提供一组实现常用数学函数（如三角函数）的方法

功能内聚

```
public class Order{
   …
   //根据订单金额和折扣规则计算需要支付的金额
   public float calculateAmountToPay(){
      …
   }
}
```

```
public class Order{
   …
   //计算订单总金额
   public float calculateTotal(){
      …
   }
   //计算折扣金额
   public float calculateDiscount(){
      …
   }
}
```

将订单金额计算方法分解为两个分别计算订单总金额和折扣金额的方法，由此得到的两个更小的方法显然更符合功能内聚的要求

实现功能内聚的方法职责更单一，关注点更聚焦，也更容易维护

如果订单折扣规则发生变化，那么开发人员只需要理解一个更小的折扣金额计算方法并作出修改。
此外，实现功能内聚的方法也更容易被复用，例如另一个项目如果订单折扣计算方式不同那么仍然可以复用订单总金额计算方法

通信内聚

```
public class ShoppingCart{
  private ArrayList shoppingList;
  private User currentUser;
  …
  //添加商品
  public void addProduct(Product product){
    …
  }
  //移除商品
  public void removeProduct(Product product){
    …
  }
  //修改商品数量
  public void changeProductNum(Product product,
    int number){
    …
  }
  //读取当前用户可用优惠券列表
  public Coupon[] getCoupons(){
    …
  }
}
```

```
public class ShoppingCart{
  private ArrayList shoppingList;
  …
  //添加商品
  public void addProduct(Product product){
    …
  }
  //移除商品
  public void removeProduct(Product product){
    …
  }
  //修改商品数量
  public void changeProductNum(Product product,
    int number){
    …
  }
}
```

左边的购物车类包含一组访问购买商品列表的方法（即添加商品、移除商品、修改商品数量），但同时也包含一个依赖于用户类的读取用户可用优惠券列表的方法，因此不符合通信内聚的要求

原来的设计还导致了购物车类与用户类之间不必要的耦合：无论用户是否有优惠券，购物车类都只需要记录并管理购买商品列表即可

常见的耦合形态-1

耦合形态	含义	建议	例子
内容耦合	一个模块具有偷偷修改另一个模块内部数据的能力，因此对其内部的内容产生了耦合	避免出现	一个类通过公开方法将内部数组引用提供给另一个类，使其具有修改数组内容的能力
共用耦合	多个模块共同访问同一个全局变量，造成这些模块之间以及与定义全局变量的模块之间的耦合	严格限制使用	一个类声明了public static变量（属性），多个其他类访问该变量。此时这个变量声明的改变或者任何一个类对于该变量的操作方式发生变化都有可能影响其他依赖于该变量的类
控制耦合	一个过程（方法或函数）通过传入“标识位”或“命令”的方式调用另一个过程并对其执行过程进行了相应控制	尽量利用多态机制消除	一个学生注册方法根据传入的学生类型进行判断，然后分别对本科生和研究生执行不同的注册过程
印记耦合	一个应用类（即实现特定应用逻辑的类）被用作一个方法的参数类型，该方法由此具有访问类的全部公开方法的能力	尽量将方法参数缩小为接口、抽象类或简单数据项	一个发送邮件的方法使用一个用户类作为参数向其发送邮件通知，而事实上该方法只需要利用用户类中的邮箱信息

常见的耦合形态-2

内聚形态	含义	建议	例子
数据耦合	一个方法具有多个基本数据类型或简单类（如String）参数，导致调用该方法的类需要准备很多数据	尽量减少参数数量	一个学生综合信息打印方法要求学生学号、姓名、院系、年级、学分数、课程成绩列表等十多个不同属性
过程调用	一个过程（方法或函数）调用另一个过程	将重复出现的调用序列封装成高层过程。	一个方法依次调用分别打印表头、表格内容、表尾的三个方法来完成一个完整报表的打印
类型使用	一个类使用另一个类作为属性、方法参数或局部参数的类型	尽量将所使用的类型缩小为接口、抽象类	一个订单类包含一个类型为用户类的属性，表示订单对应的用户
文件包含和包引入	一个模块通过include语句引入一个文件或通过import语句引入一个包	消除不必要的包含和引入	一个报表打印类通过import语句引入打印机接口相关的包实现打印操作
外部依赖	一个模块对当前软件系统范围之外的系统（例如操作系统）产生依赖	尽量减少代码中的外部依赖	一个类通过Java本地化接口（Java Native Interface）调用Windows系统上的DLL文件中所实现的功能

内容耦合

```
public class CampusCard{
   //消费记录数组
   public Consumption [] consumpList;
}
```

```
public class CampusCard{
   //消费记录数组
   private Consumption [] consumpList;
   //获取消费记录条数
   public int getConsumpNum(){
      …
   }
   //按照指定序号返回一条复制的消费记录
   public Consumption getConsumptionAt(int index){
      …
   }
}
```

左边的设计方案中就隐含着内容耦合

消除内容耦合的主要手段是实现信息隐藏
正如右边的设计方案那样，通过将内部数据隐藏起来并通过受控的接口对外提供访问，其他类除了按照预定的方式访问数据外不再具有修改其内部数据的能力了

共用耦合

- 由于共享全局变量的使用而产生
  - ✓ 在结构化程序（例如C语言程序）中使用较多
  - ✓ 在面向对象程序（例如Java程序）中使用较少（应当尽量避免使用public static变量）
- 共用耦合的危害主要在于其影响面、隐藏性和不可控性
  - ✓ 共享同一全局变量的模块可能很多，因此影响面大
  - ✓ 共享同一全局变量的多个模块之间的相互影响不像调用关系那样明显，因此经常会被忽略
  - ✓ 全局变量的使用方式缺少必要的控制，容易造成意想不到的副作用

控制耦合

```
public void drawFigure(Shape[] shapeList){
  …
  for(int i=0;i<shapeList.length();i++){
    //如果是圆形
    if(shapeList[i].type.equals("circle")){
      …
    }
    //如果是矩形
    if(shapeList[i].type.equals("rectangle")){
      …
    }
  }
}
```

```
public void drawFigure(Shape[] shapeList){
  …
  for(int i=0;i<shapeList.length();i++){
    //调用图形类的抽象绘制方法
    shapeList[i].draw();
  }
}
```

```
public abstract class Shape{
  //图形绘制的抽象方法
  public abstract void draw();
  …
}
```

```
public class Circle extends Shape{
  //重写父类的图形绘制方法，提供针对圆形的具体实现
  public void draw(){
    …
  }
  …
}
```

左边的设计方案中存在控制耦合，带来的问题是，drawFigure方法需要不断根据图形种类的变化而进行修改

可以通过引入多态（polymorphism）来消除这种控制耦合

如图右边的设计方案，通过将图形类变成抽象类并声明抽象方法draw，我们可以建立图形类的继承层次，同时利用多态机制直接调用抽型的图形类的绘制方法而不需要对具体类型进行判断

其他耦合

其他耦合形态（即印记耦合、数据耦合、过程调用、类型使用、文件包含和包引入、外部依赖）属于常规的耦合形态，基本的处理方针就是尽量降低耦合度

```
//向指定用户发送邮件消息
public void sendEmail(User user, String msg){
  …
}
```

```
//向指定地址发送邮件消息
public void sendMail(String address, String msg){
  …
}
```

```
//向指定地址发送邮件消息
public void sendMail(Address address, String msg){
  …
}
```

左边的设计方案中存在印记耦合，邮件发送功能只需要有邮件地址即可，并不需要用户的其他信息

两种改进的设计方案：采用字符串类型（如果地址只是一个简单的字符串）来取代原来的用户类；采用专门的地址类型（如果地址本身也是一个包含多种信息的对象）来取代原来的用户类

这两种改进方案都降低了原来的印记耦合度，并使得发送邮件的方法不再依赖于用户类，从而使其具有更广泛的复用性

面向对象设计原则

- 面向对象软件设计的一个主要目标是可维护性和可扩展性，即软件设计及实现代码容易理解、修改以及扩展新功能和新特性
- 基于以上设计目标，面向对象软件设计形成了一系列被称为SOLID的设计原则
  - ✓ 单一职责原则（Single Responsibility Principle）
  - ✓ 开闭原则（Open Closed Principle）
  - ✓ 里氏替换原则（Liskov Substitution Principle）
  - ✓ 迪米特法则（Law of Demeter）
  - ✓ 接口隔离原则（Interface Segregation Principle）
  - ✓ 依赖转置原则（Dependence Inversion Principle）

单一职责原则

- 每个**类、接口、方法**都应该只具有单一的职责，应该只会因为一个原因发生变化
- 强调的是类、接口、方法的内聚性
- 内聚性越高，它们的职责越单一，发生变化的原因也越集中
- 类与方法相比粒度更大，可以尽量让一个类具有更少、更加聚焦的职责

```
public class Order{
   …
   //根据订单金额和折扣规则计算需要支付的金额
   public float calculateAmountToPay(){
      …
   }
}
```

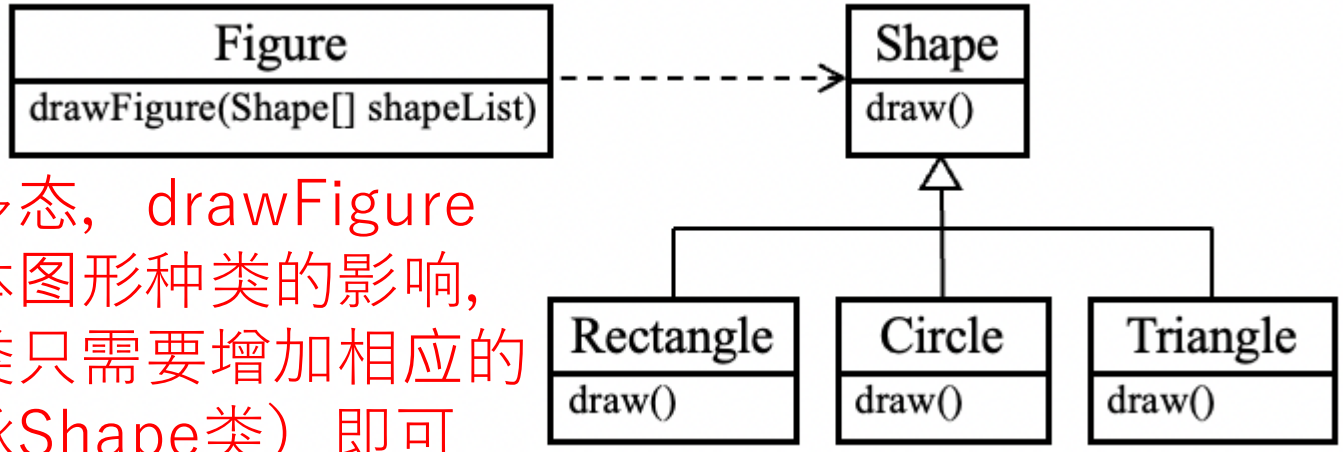
订单总金额计算和折扣金额计算两部分中任何一个发生变化都有可能导致这个方法变化

```
public class Order{
   …
   //计算订单总金额
   public float calculateTotal(){
      …
   }
   //计算折扣金额
   public float calculateDiscount(){
      …
   }
}
```

实现了功能内聚

开闭原则

- 一个模块应该对扩展开放对修改封闭
  - ✓ 软件增加新功能或新特性时应当通过扩展新的代码单元（如类、方法）而非修改已有的代码单元的方式来实现
  - ✓ 理解和修改现有代码的过程中面临较大的复杂性挑战，工作量较大且容易引入代码缺陷
- 一个重要而且基础性的设计原则，直接体现了软件设计的可扩展性的目标
- 关键在于引入适当抽象并为未来扩展留下空间



通过引入继承及多态，drawFigure方法不再受到具体图形种类的影响，增加新的图形种类只需要增加相应的图形子类（即继承Shape类）即可

里氏替换原则

- 子类对象可以出现在任何父类对象出现的地方
  - ✓ 为了确保子类满足父类对外的所有约定（或称为契约）
  - ✓ 这样按照父类对外声明的行为与之交互的其他类在遇到子类的对象时才不会发生问题
  - ✓ 诸如多态这样的机制才能确保有效实现
- 在设计继承关系时需要按照里氏替换原则验证父类与子类之间的关系，如果不符合则应该避免使用继承关系

正方形类增加了一个隐含的约束，即宽度和高度相等
这样，在一些宽度和高度设置为不同数值的地方，矩形对象可以出现而正方形对象则不能

```
public class Rectangle{
   private double width;
   private double height;
   public void setWidth(double width);
   public void setHeight(double height);
   public double getArea();
   …
}
```

```
public class Square extends Rectangle{
   …
}
```

迪米特法则

- 强调不要和“陌生人”说话，只与直接“朋友”交谈
  - ✓ 每个类的直接“朋友”包括作为其属性、方法参数或返回值的对象
  - ✓ 尽可能只与直接的“朋友”打交道，不要再跟其他类的对象打交道，例如在方法内引入其他类的对象作为局部变量
- 出发点是限制一个类与其他类的依赖关系：尽量通过已有的联系满足所需要的请求，而不要再引入新的类依赖

```
//展示一本图书的信息
public void displayBook(Book book){
   …
   //展示图书作者信息
   Author author=book.getAuthor();
   display(author.getName());
   …
}
```

displayBook方法为了获取图书作者信息而在方法内部引入了对Author类的依赖，从而违反了迪米特法则

```
//展示一本图书的信息
public void displayBook(Book book){
   …
   //展示图书作者信息.
   display(book.getAuthorName());
   …
}
```

Book类直接提供获取作者姓名的方法，displayBook方法无需与Author类交互

接口隔离原则

- 多个服务于特定请求方的接口好过一个通用接口，不要强迫一个类依赖它不会使用的接口
- 使用“万能”接口会导致使用这种接口的类被迫接受一些并不需要的操作，从而增加耦合性以及开发人员理解接口的负担



避免瑞士军刀式的“万能”接口

左边的Shape接口包含了两种不同的图形相关职责，即支持几何计算的抽象图形和支持图形绘制的可视化图形

```
public interface Shape{
   //获取边的数量
   public int getNumOfSides();
   //获取第i条边的长度
   public double getLenOfSide(int i);
   //绘制图形
   public void draw();
   //获取图形面积
   public double getArea();
}
```

```
public interface AbstractShape{
   //获取边的数量
   public int getNumOfSides();
   //获取第i条边的长度
   public double getLenOfSide(int i);
   //获取图形面积
   public double getArea();
}
```

```
public interface DrawableShape{
   //获取边的数量
   public int getNumOfSides();
   //获取第i条边的长度
   public double getLenOfSide(int i);
   //绘制图形
   public void draw();
}
```

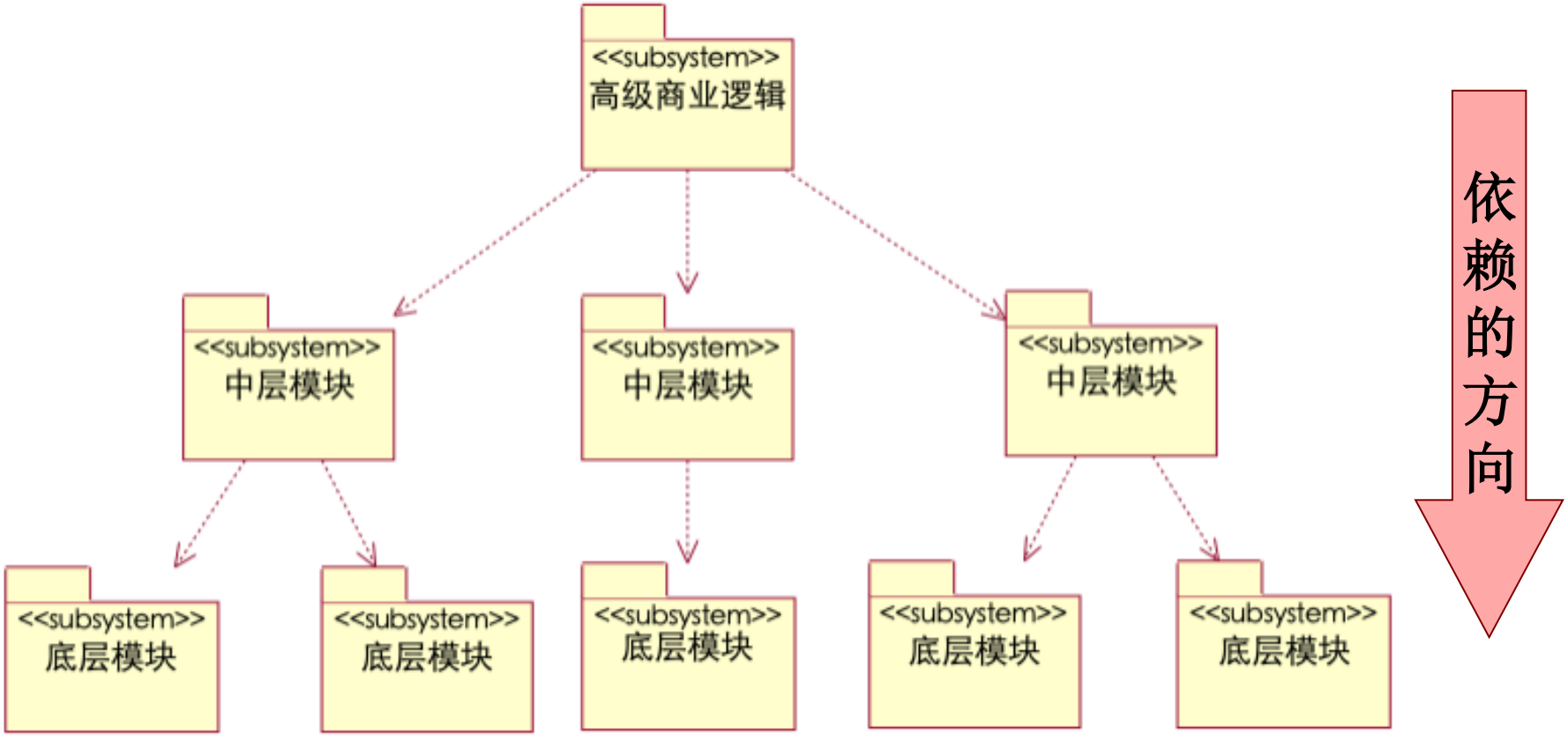
这种“通用”接口一方面不内聚，另一方面增加了接口实现和使用的复杂性和依赖性（例如引入支持图形绘制的图形库）

依赖转置原则

- 强调应该尽量依赖于抽象（例如抽象类、接口）而非具体（例如具体的实现类）
- 高层模块不应该依赖于低层模块。二者都应该依赖于抽象
- 抽象不应该依赖于细节。细节应该依赖于抽象
- 针对接口编程，不要针对实现编程
- 依赖于抽象还有利于长期的演化和维护
  - ✓ 抽象的东西不容易变化，依赖于抽象的其他类受到变化的影响较小
  - ✓ 抽象依赖为未来的扩展提供了方便，如果有新的实现方式，那么只要实现同样的接口，从而实现开闭原则的要求

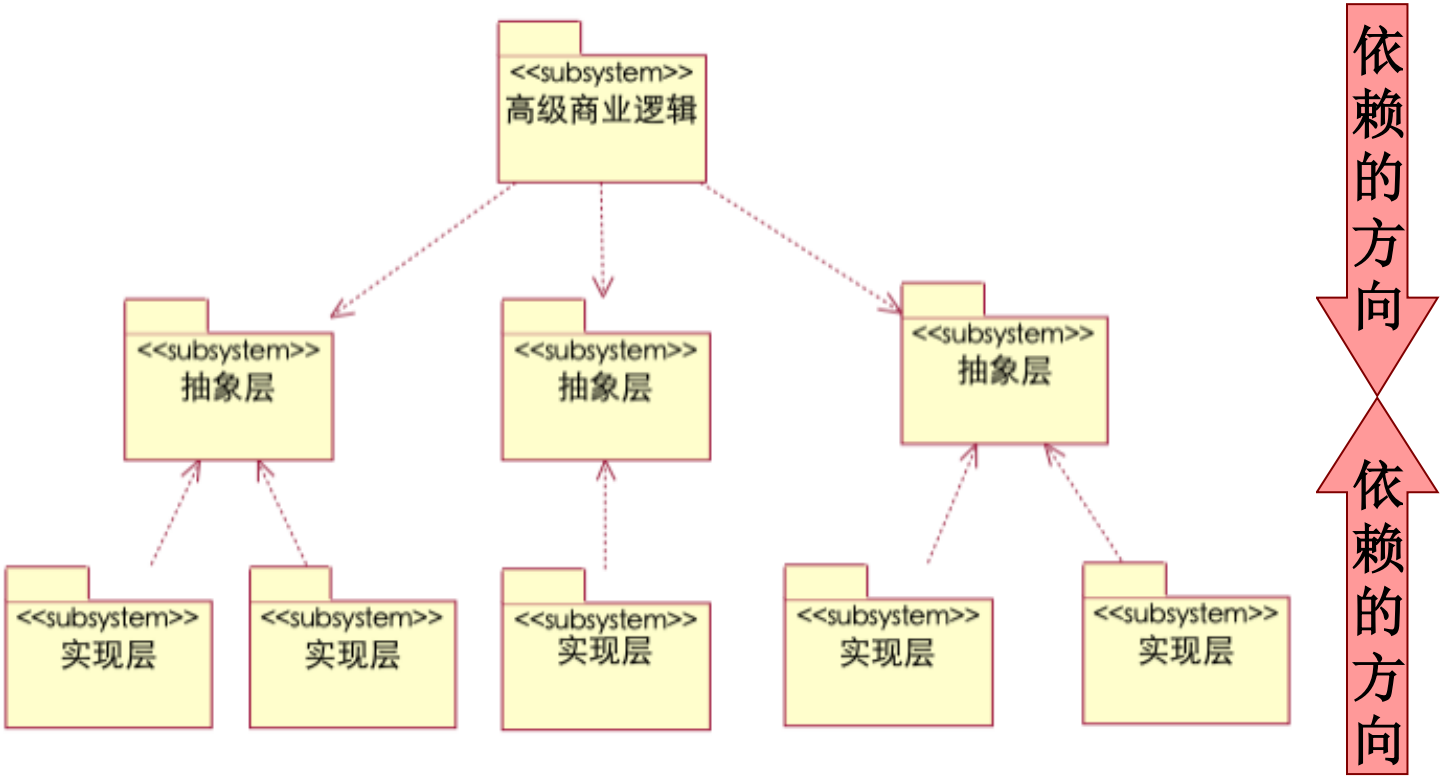
依赖转置原则

传统的依赖关系



依赖转置原则

符合DIP的系统



依赖转置原则

启发式原则

- “依赖于抽象” —程序中所有依赖关系都应该终止于抽象类或者接口
- 启发式原则：
  - ✓ 任何变量都不应该拥有指向具体类的指针或者引用
  - ✓ 任何类都不应该从具体类派生

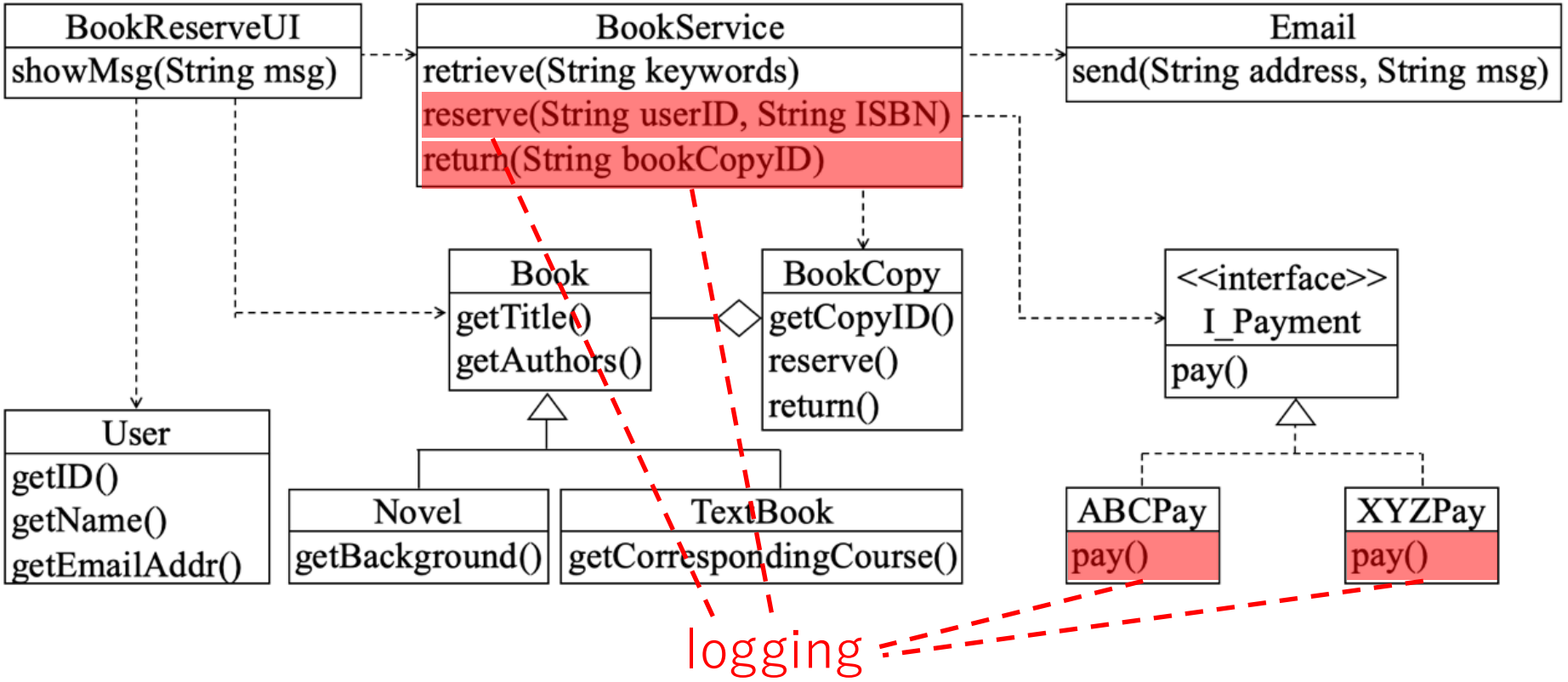
面向切面的编程（AOP）

- 面向对象软件设计以包、类和方法实现层次化的模块分解以及相应的数据和操作封装，以关注点分类作为基本的设计思想
- 这种单一维度的模块化机制的问题
  - ✓ 代码交织（code tangling）：与多个关注点相关的实现代码混杂在一个模块中
  - ✓ 代码散布（code scattering）：与同一个关注点相关的实现代码分散在多个模块中
- 面向切面的编程方法支持横切关注点解耦和模块化封装，为进一步改进软件设计提供了支持

AOP：Aspect-Oriented Programming

横切关注点（Crosscutting Concern）

图书馆管理子系统静态结构设计（UML类图）

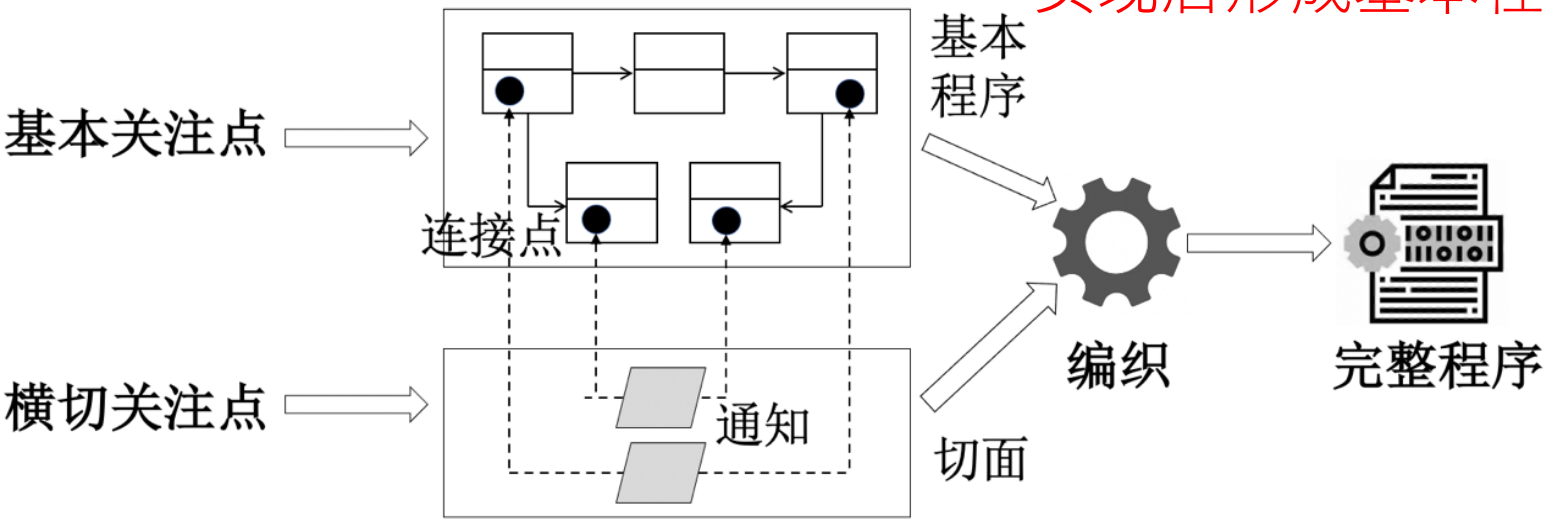


多个类和方法中都需要在完成特定操作后进行日志记录，这一横切关注点影响多个类和方法，导致与日志记录相关的关注点（例如日志规范）与自身的业务关注点（例如图书服务、支付等）交织在一起，同时与日志记录关注点相关的代码也散布在多个类中

面向切面编程机制

开发人员在基本程序上定义连接点，相关工具自动将切面编织到对应的连接点上并形成最终的完整程序

与业务相关的基本关注点按照面向对象软件设计方法产生基于类的模块化分解结构，实现后形成基本程序



横切关注点单独以“切面”这种新引入的模块化机制来进行模块化封装和实现

最终基本关注点和横切关注点的实现代码通过编织实现了最终集成

面向切面编程相关概念-1

- 切面（Aspect）：实现关注点的模块化单元
  - ✓ 一些实现功能逻辑的代码（称为通知）
  - ✓ 在哪些地方、什么时候、以何种方式调用这些代码的指令
- 通知（Advice）：由切面所封装的、将插入到基本程序中指定地方执行的功能代码
  - ✓ 例如，实现日志记录的功能代码就是一个通知
  - ✓ 通知可以被插入到程序中不同的地方，例如目标连接点（如方法调用）之前、之后、周围（即之前和之后），或者替换连接点处原有的代码
- 连接点（Join point）：基本程序结构或者执行流上的一种可以明确定义的位置
  - ✓ 在这些位置上可以插入切面所定义的新的行为
  - ✓ 常见的包括方法调用、属性的定义和访问、异常抛出等，例如I_Payment接口的pay方法调用可以定义为连接点

面向切面编程相关概念-2

- 切点指示器（Pointcut Designator）：为切面的定义提供了一种批量描述一系列连接点上的插入位置的机制
  - ✓ 一般都是通过使用全称量词来实现
  - ✓ 例如，可以要求在某个包中所有类的公开方法被调用之前插入日志记录，或者要求实现某个接口的所有类的公开方法被调用之前插入日志记录
- 编织（Weave）：将基本程序与切面集成到一起获得完整程序的过程
  - ✓ 可以通过静态方法将切面代码与基本程序代码组合在一起进行编译
  - ✓ 也可以在程序装载运行时动态地插入切面代码，还可以通过修改编译器来实现切面的插入执行

面向切面编程实现方式

- 借助于具体的语言或框架来实现，包括
  - ✓ 定义切面（包括相关的通知实现和连接点、切入点定义等）的语言和编程规范
  - ✓ 切面编织的实现机制
- 针对主流语言的AOP语言扩展及编程框架：AspectJ、AspectWerkz、JBoss AOP、Spring AOP、AspectSharp、AspectC++等
- Spring AOP
  - ✓ Spring（面向Java的软件开发框架）内置的AOP支持
  - ✓ 依托Spring的核心IOC（即Inversion of Control，控制反转）容器来实现：容器根据所配置的规则创建对象并通过依赖注入的方式将对象实例传入程序中
  - ✓ 通过运行时的Java代理机制实现切面编织

Spring AOP所支持的切面类型

切点注解	切面含义
@Before	在目标方法调用之前插入通知中的实现代码
@After	在目标方法执行完成并返回前（无论正常返回或异常退出）插入通知中的实现代码
@AfterReturning	在目标方法调用正常返回之后插入通知中的实现代码（可以获得返回值并进行处理）
@AfterThrowing	在目标方法调用抛出异常之后插入通知中的实现代码（可以获得所抛出的异常信息）
@Around	在目标方法调用之前和之后插入通知中的实现代码（可以决定目标方法是否执行）

只支持与方法调用相关的连接点，所支持的切面类型也都是与方法调用相关的

通过使用Java注解（annotation）机制来进行各种切面的声明

基于Spring AOP的切面定义

```
@Aspect
@Component
public class PayAspectJ {

  @Pointcut("execution(* cn.setextbook.examples.I_Payment.pay(..))")
  public void point(){}

  @Before("point()")
  public void logPayment() {
    logger.info("start an online payment");
  }

  @Around("point()")
  public void checkPermission(ProceedingJoinPoint jp) {
    if(checkPermission())
      jp.proceed();
    else
      logger.info("invalid payment");
  }
}
```

切点表达式明确了I_Payment接口中所有名为pay的方法都在覆盖范围内

定义的切点（point方法）表示I_Payment接口的pay方法的调用

在pay方法执行之前插入一个日志记录方法

在pay方法执行之前进行许可检查，只有检查通过才会正常执行pay方法（通过调用proceed方法），否则不执行pay方法并记录日志

Spring AOP也支持利用配置文件进行切面的定义

基本程序（例如支付相关模块）的开发人员无需了解这些切面，切面的开发与基本程序的开发独立进行

面向对象软件设计中的约定

- 面向对象软件设计中的约定
  - ✓ 支持不同类的开发者在明确类职责的基础上独立进行开发
  - ✓ 保证不同的类分别开发完成后能够顺利集成并正确实现整体系统设计要求
- 基于接口定义的设计约定
  - ✓ 类之间的接口约定主要依赖于接口方法的语法声明，包括参数及其类型、返回值类型、抛出的异常等
  - ✓ 这种语法声明不足以精确定义类之间的接口语义，从而埋下了接口理解不一致的隐患
- 接口定义中的模糊性带来的问题
  - ✓ 还会导致出错之后难以定位问题根源以及厘清责任
  - ✓ 接口的实现方需要承担更多的责任对各种潜在的问题（例如不合法的参数）进行检查，内部逻辑过于复杂，甚至调用链上的一系列代码单元重复对某些问题进行检查

接口定义中的模糊性

1）支持哪些运算操作不清楚
2）方法的操作符参数（operator）的精确定义不清楚
3）谁应该对除0操作进行检查不清楚

```
//根据指定操作符进行计算并返回结果
public int calcualte(int operator,
 int a, int b){
  switch(operator){
  case 1:
    return a+b;
  case 2:
    return a-b;
  case 3:
    return a*b;
  case 4:
    return a/b;
  }
  return 0;
}
```

```
public class IntStack{
  private int[] elements;
  private int topIndex;
  //构造方法按照指定容量构造并初始化堆栈
  public IntStack(int capacity){
    elements=new int[capacity];
    topIndex=-1;
  }
  //返回堆栈是否为空
  public boolean isEmpty(){
    return topIndex<0;
  }
  //执行出栈操作
  public int pop(){
    int top=elements[topIndex];
    topIndex= topIndex-1;
    return top;
  }
  …
}
```

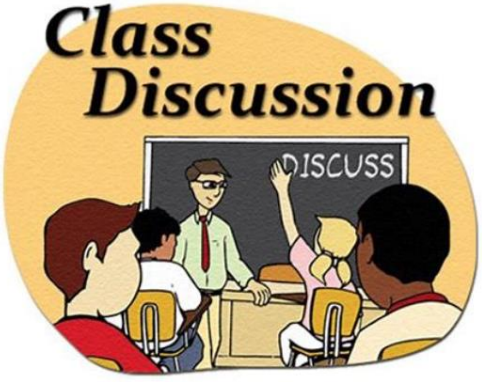
各种“假定”没有被明确定义出来，使用这个类的开发者可能会根据自己的错误理解进行编码，从而造成问题

pop方法没有对当前堆栈是否为空进行检查，这个类的开发者“假定”调用方应该在调用pop方法之前先通过isEmpty方法确认堆栈不为空

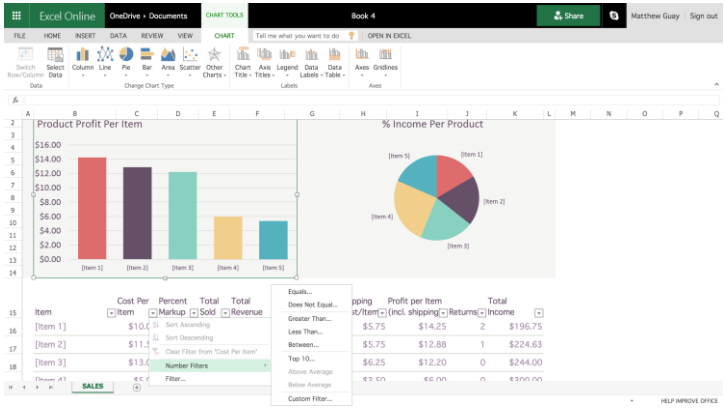
设计模式（Design Pattern）

- “模式”这一概念在很多领域中都有所体现，一般用于表示某种可借鉴的参考解决方案
- 在软件设计领域，设计模式是指针对一类相似设计问题的通用和参考性的设计方案，一般都经过大量的实践验证，能够较好地实现相关的设计目标
- 有经验的软件开发人员一般都能熟练掌握一些常用的设计模式并能够结合具体的开发要求进行实例化的应用

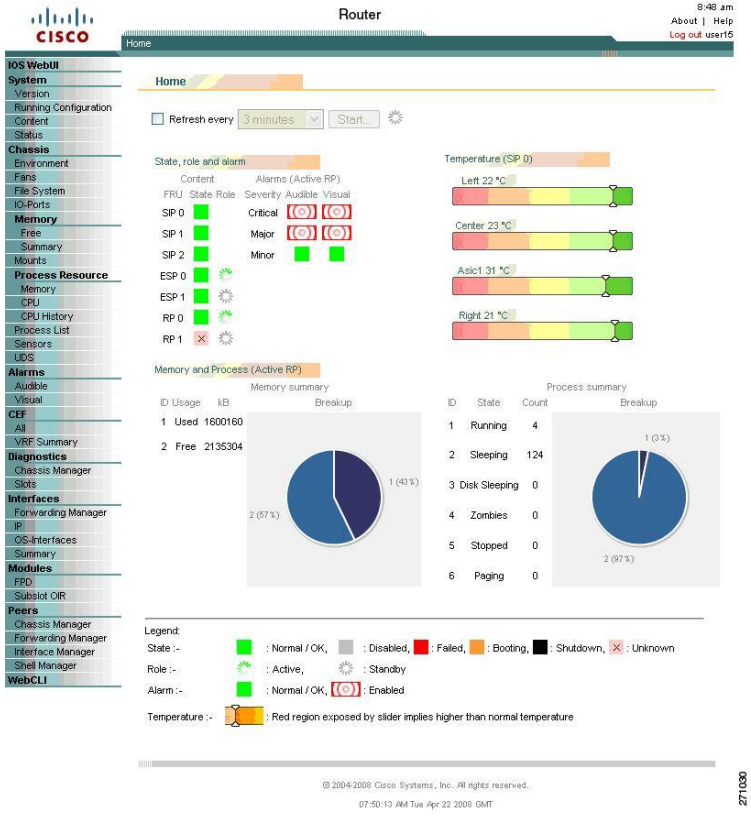
课堂讨论：软件设计中的共性



课堂讨论：举例具有共性（功能、方案等）的软件或软件系统





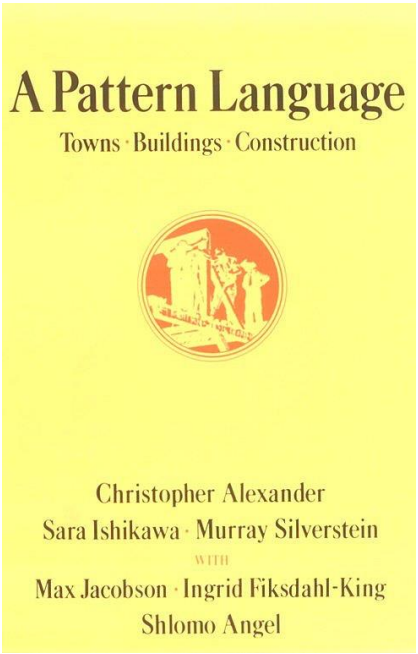


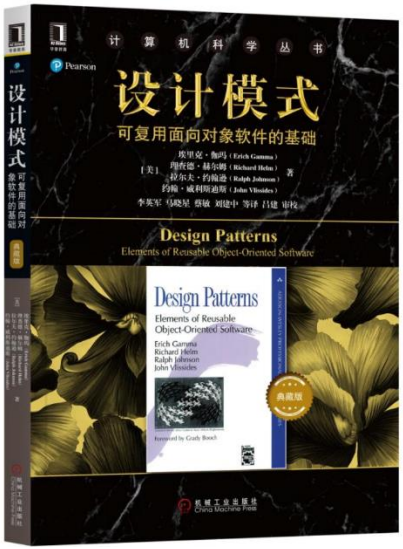
共性设计问题的参考设计方案

- 软件应用中存在很多共性设计问题
- 这些设计问题的具体形式有所差异，但可抽象为共性问题
- 共性设计问题存在良好的参考设计方案：易理解、易修改、易扩展
  - ✓ 优化职责分配：例如MVC模式中，视图不再具备数据更新或控制职责，更加符合单一职责的原则
  - ✓ 关键是解耦：例如MVC模式中，视图之间解耦、模型不依赖于控制器或视图
  - ✓ 重要手段是引入抽象：例如MVC模式中，观察者接口的引入隔离了模型对控制器和视图的依赖

设计模式的起源和意义

- 从Christopher Alexander关于建筑设计模式的思想中派生而来
- Gamma等人（被称为四人组）的著作使得面向对象设计模式正式进入人们的视野
- 设计模式的意义
  - ✓ 经过验证的共性设计问题解决方案
  - ✓ 谈论设计的通用词汇表





JHotDraw（http://www.jhotdraw.org）：一个包含丰富的设计模式实例的开源Java GUI框架

Hillside小组（hillside.net/patterns）：设计模式资源汇编

面向对象设计模式

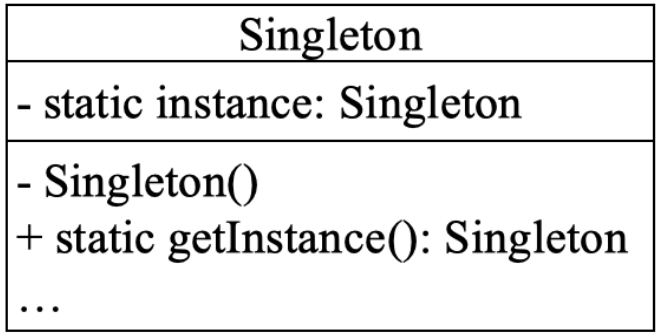
- Gamma等人总结了23种常用的面向对象设计模式及其基本要素
  - ✓ 名称：有意义的、能揭示设计模式目的的名称
  - ✓ 问题：关于设计模式所针对的问题域描述，解释了该模式何时适用
  - ✓ 解决方案：对于设计解决方案的描述，包括各个组成部分、各自的职责以及相互之间的关
  - ✓ 使用效果：应用这种设计模式的效果，包括可能的结果以及多方面因素的权衡，帮助使用者决定是否使用这种模式
- 设计模式是一种抽象的解决方案，在应用时需要结合具体问题和上下文进行实例化

面向对象设计模式分类

模式分类	分类含义	所包含的设计模式
创建型模式	与类的实例对象创建相关的设计模式，关注于对象创建过程的抽象和封装	工厂方法（Factory Method）、抽象工厂（Abstract Factory）、单例（Singleton）、建造者（Builder）、原型（Prototype）
结构型模式	与类和对象的结构组织相关的设计模式，关注于如何实现对象的组合	适配器（Adaptor）、装饰器（Decorator）、代理（Proxy）、外观（Facade）、桥接（Bridge）、组合（Composite）、享元（Flyweight）
行为型模式	与类和对象之间的交互行为和通信相关的设计模式，关注于类和对象之间的交互关系和职责分配	策略（Strategy）、模板方法（Template Method）、观察者（Observer）、迭代器（Iterator）、责任链（Chain of Responsibility）、命令（Command）、备忘录（Memento）、状态（State）、访问者（Visitor）、中介者（Mediator）、解释器（Interpreter）

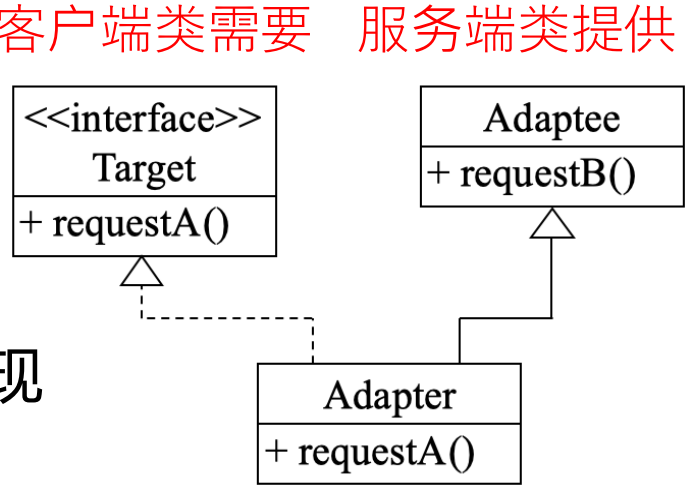
单例模式

- 应用场合
  - ✓ 一个类只允许有一个对象实例
  - ✓ 例如操作系统中的任务管理器和回收站、网站的计数器、金融交易的引擎、应用程序的日志引擎等
- 解决方案考虑
  - ✓ 要求所有需要访问这个类的对象的地方都只能获取到同一个对象引用，而不能创建不同的对象
  - ✓ 可以通过定义类的静态方法来实现，但这样就无法使用接口、继承等面向对象机制了
- 参考设计方案
  - ✓ 将构造方法设为私有
  - ✓ 内部包含一个类型为自身的静态成员对象（instance）
  - ✓ 通过一个静态方法（getInstance）用于获取这个成员对象的引用



适配器模式

- 应用场合
  - ✓ 一个类所需要的接口与另一个类所提供的接口不匹配，但功能相同或相近
  - ✓ 此时可以通过适配器在二者之间进行转换和适配
- 参考设计方案
  - ✓ 额外引入的适配器类（Adapter）一方面继承了Adaptee从而具备了该类所实现的能力，另一方面则实现了Target接口
  - ✓ 客户端类创建Adapter类的对象实例后可以将其作为接口Target的对象实例进行使用
  - ✓ Adapter类利用继承所得到的能力实现所需的方法，实现转换和适配
  - ✓ 也可利用委托关系来实现适配器，即适配器内部包含Adaptee类型的成员对象，通过该对象的方法来实现所需要的能力

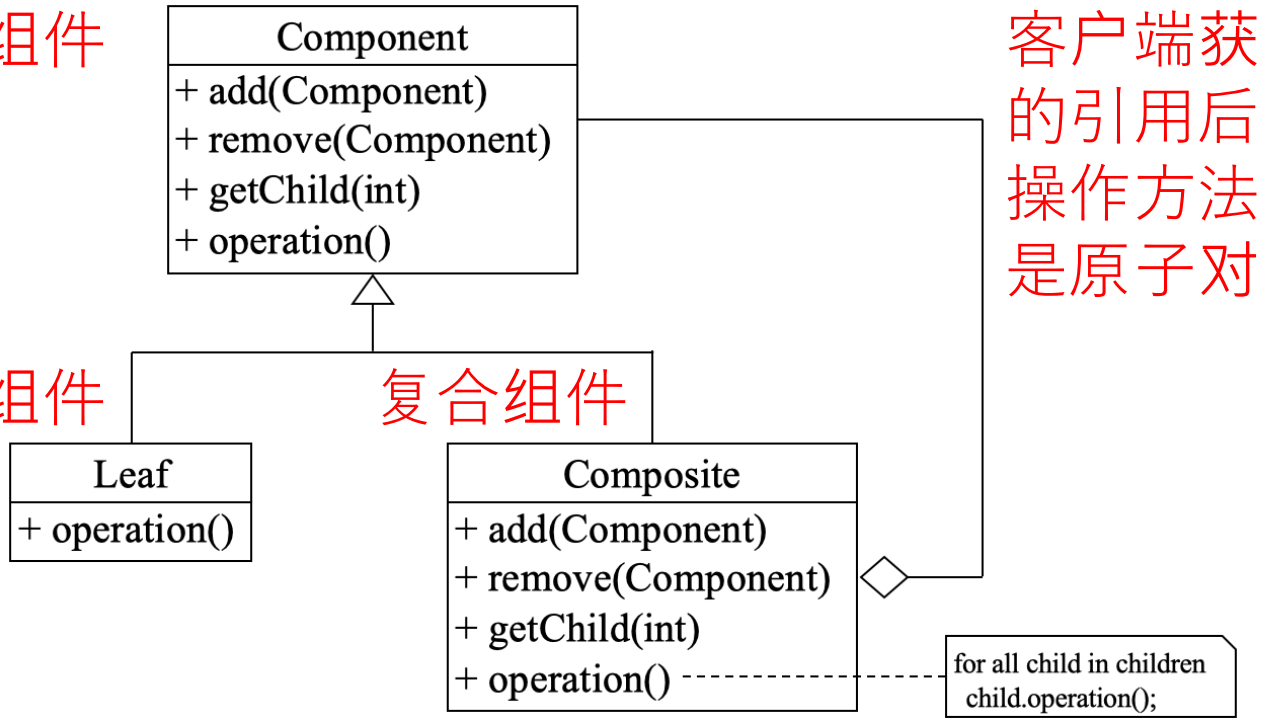


组合模式-1

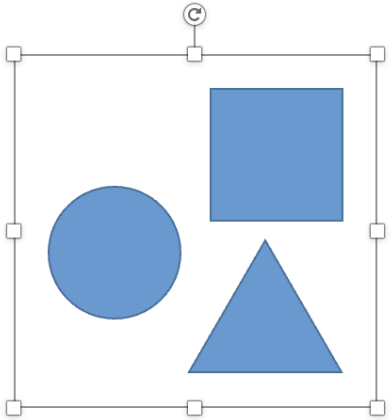
- 应用场合
  - ✓ 需要处理的对象构成了一种表示整体部分层次的树状结构，例如文件系统中嵌套的目录结构、画图工具中嵌套的复合图形
  - ✓ 客户端开发人员希望以一种统一的方式处理对象以及对象的组合，从而避免在代码中针对不同类型对象采取不同的处理策略（例如，对于原子对象直接调用操作，而对于复合结构则需要遍历其所包含的对象或下一级复合结构进行处理）
- 参考设计方案
  - ✓ 一方面允许将对象组织成嵌套层次结构，另一方面允许客户端代码以统一的方式处理对象以及对象的组合
  - ✓ 复合对象可以包含一组子对象，其中既可以有原子对象又可以有下一级的复合对象
  - ✓ 抽象的组件对象类上定义了某种操作方法operation
  - ✓ 原子对象可以直接给出方法的实现，而复合对象则进一步调用所包含的下一级原子对象或复合对象的同一操作方法来实现

组合模式-2

抽象组件



客户端获得抽象组件对象的引用后可以直接调用其操作方法，而不用关心它是原子对象还是复合对象



例如，在画图工具中的复合图形绘制中

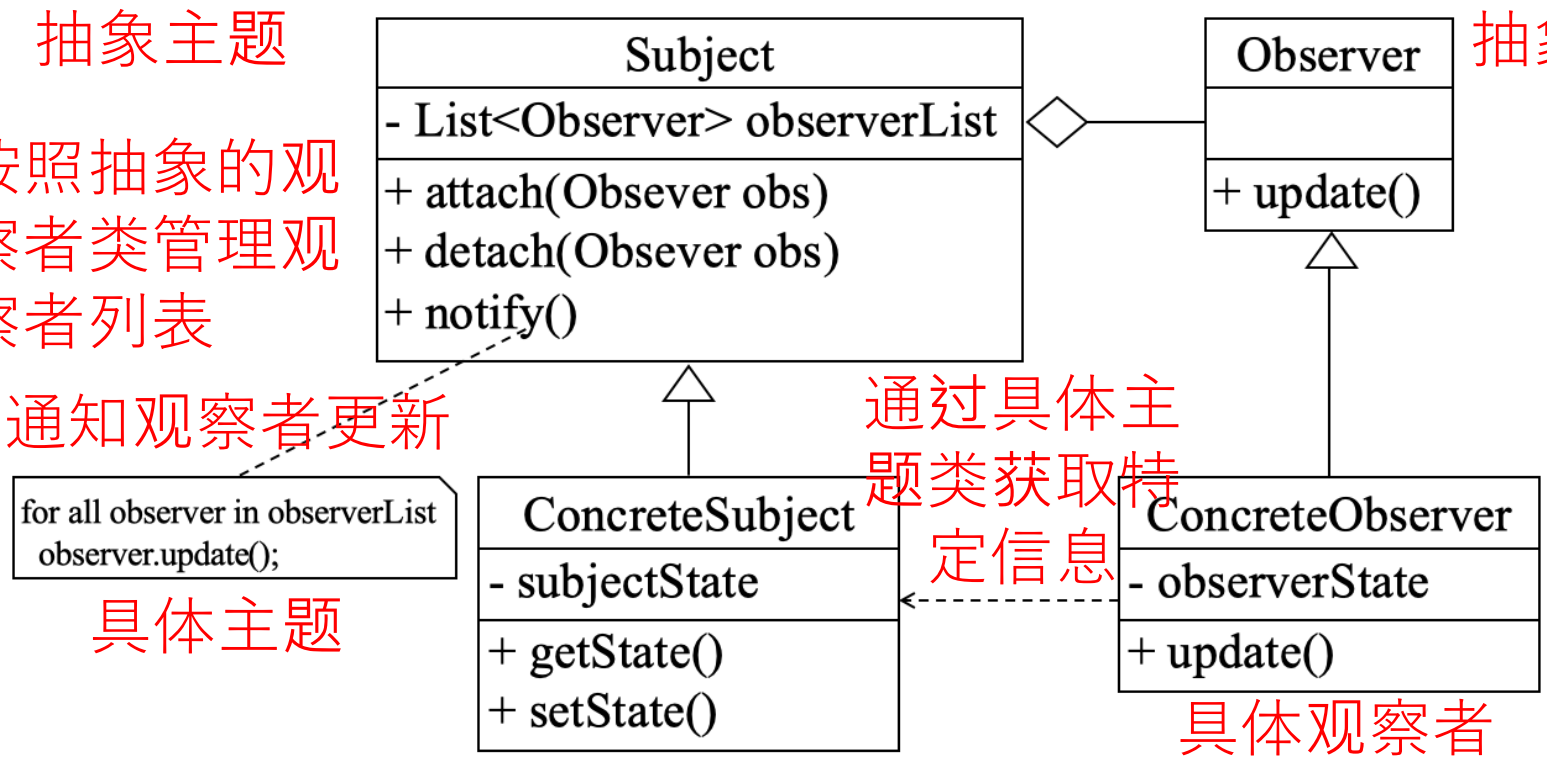
- ✓ 抽象的组件对象是抽象图形
- ✓ 原子对象是基本图形（三角形、圆形等，可以有多个）
- ✓ 复合对象是复合图形

客户端代码获取一个表示图形复合结构的根结点对象引用后，可以按照抽象图形调用其操作（例如绘制、移动、删除），从而实现整个图形的操作，而不用关心其是一个原子图形还是复合图形

观察者模式-1

- 应用场合
  - ✓ 在包含用户界面的应用程序中，经常存在同样的数据对象通过多种不同的形式进行可视化展示的情况，例如关于道路交通的实时监控数据可以通过地图、表格以及各种统计图表等不同的形式来展现
  - ✓ 数据的状态发生变化后各个展现视图都需要随之更新
  - ✓ 各个视图都有可能发生变化，未来还有可能增加新的展现方式
  - ✓ 按照单一职责原则以及开闭原则，我们希望数据管理与数据展现相分离，同时新增展现视图无需修改已有的代码
- 参考设计方案
  - ✓ 负责数据管理的主题类（Subject）与负责显示逻辑的观察者（Observer）相分离
  - ✓ 观察者通过对主题（即被观察者）的观察来实现及时更新
  - ✓ 主题类中包含当前观察者的列表，当数据状态发生变化时可以调用观察者的通知方法（notify）来通知它们进行更新

观察者模式-2



观察者模式通过主题类与具体观察者类的解耦实现了对于具体观察者类扩展的支持，但这其中所引入的设计抽象也有一些不利影响。例如，主题对象的状态变化将导致所有具体观察者对象的更新操作，而其中一些更新可能是不必要的，因为对应的具体观察者可能并不关注于所做的更新。

演化式设计

- 计划驱动的开发过程（以瀑布模型为代表）
  - ✓ 经典软件工程信奉的是在各种工程领域中广泛采用的计划驱动的开发过程
  - ✓ 要求设计师在实现开始前做好高层设计并逐步细化，然后将详细的设计方案交给另一组开发者进行实现
  - ✓ 类似于建筑工程中设计师画好图纸然后交给建筑工人施工
- 迭代和演化式的开发过程（敏捷开发）
  - ✓ 通过短周期的迭代（例如一到两周）不断交付可运行的软件从而持续获得用户反馈并不断调整开发计划
  - ✓ 认为长期的计划具有很强的不确定性，因此强调以迭代化的方式制订短期计划并根据变化和反馈不断进行调整
  - ✓ 倾向于采用所谓的演化式设计（Evolutionary Design），即设计随着实现过程的进展而逐步发展，而设计师的角色在一定程度上与开发者相融合

Martin Fowler对于演化式设计的观点

计划设计无法对软件实现过程中可能遇到的所有问题进行预见和预判，从而导致实现过程中开发者对设计产生质疑。软件设计人员如果总是忙于考虑软件设计而不参加具体的编码活动，那么软件技术的快速发展可能很快会导致软件设计人员失去编码人员的尊重。与之相对比，建筑业中设计人员和施工人员的技能界限很明确，施工人员一般不会质疑建筑设计方案，而软件开发人员很容易会优秀到足以质疑设计人员给出的设计方案的程度。此外，软件开发中的需求变更几乎总是不可避免的，这就要求软件设计必须要有灵活性，而这对于事先计划的设计是一个很大的挑战。

演化式设计中设计师和开发者角色的融合以及迭代式的设计方案演化调整很大程度上可以解决以上这些问题。然而，演化式设计不提倡太多的预先设计，而是鼓励在迭代式的演化过程中逐步考虑相关设计问题，这使得设计决策似乎主要来自于开发过程中一堆即兴的决定。预先设计的缺乏有可能会使软件越来越难以应对变化，而整个软件开发似乎回到了很久以前那种“写了再改”（Code and fix）的低效模式，从而导致修复bug的成本随着时间的推移越来越高。

《Is Design Dead?》（Martin Fowler. May 2004）
https://www.martinfowler.com/articles/designDead.html

抚平变化曲线

- 两种设计方法的问题都与软件变化曲线相关，即随着软件项目的进展修改软件的成本呈指数级增长（即时间越长修改越困难）
  - ✓ 计划设计由于预见性的不足以及需求的变化而逐渐无法适应需要，导致软件修改越来越难
  - ✓ 演化式设计由于缺乏事先设计而使软件无法为变化做好准备
- Martin Fowler建议将持续集成、测试、**重构**相结合，实现“抚平变化曲线”的目标
  - ✓ 持续集成使得不同开发人员的工作可以持续保持同步，不用担心个人所做的修改与其他人的代码出现集成问题
  - ✓ 测试可以及时暴露代码中的问题并提供反馈，同时为开发团队创造一种安全感
  - ✓ 持续和系统性的重构则可以不断将设计思考引入软件中

平衡计划设计与演化式设计

- Martin Fowler所建议的持续集成、测试、重构这三种实践已成为敏捷开发实践的重要组成部分
- 也就是说当前的敏捷开发实践已经可以在很大程度上实现“抚平变化曲线”的目标
- 在此基础上，可以考虑将计划设计与演化式设计相结合，并找到一个合适的平衡点
  - ✓ 在软件编码开始之前应当做少量的预先设计，对于一些相对稳定同时涉及不同模块开发者之间约定的设计进行决策，同时将更多的设计决策留给后续开发迭代
  - ✓ 这样在每次迭代中，开发人员在开始针对特定开发任务编码之前可以考虑是否需要通过重构引入一些设计上的考虑

代码坏味道

- 软件重构是实现演化式设计的关键，也是在迭代化开发过程中逐步引入设计决策的重要手段
- 确定软件重构的时机，即何时以及在何处开展重构，就成为一个关键性的问题
- Martin Fowler和Kent Beck提出利用代码坏味道（bad smell）这一概念形容软件重构的时机

“我们看过很多很多代码，它们所属的项目从大获成功到奄奄一息的都有。观察这些代码时，我们学会了从中找寻某些特定结构，这些结构指出（有时甚至就像尖叫呼喊）重构的可能性。”

典型的代码坏味道

名称	含义	重构建议
重复代码（Duplicated Code）	程序中不同的地方存在相同或相似的代码。	将相似或相同的代码抽取成新的方法或类，然后将原来包含它们的地方改为对所抽取代码的调用。
过长的方法（Long Method）	方法包含的代码过长。	将方法中各个相对独立的部分抽取成多个新的方法，在原方法中调用这些新方法。
过大的类（Large Class）	一个类所承担的职责过多，例如其中包含过多的对象属性。	将类中各组密切相关的对象属性抽取到多个新的类中。
过长参数列表（Long Parameter List）	一个方法所要求的传入参数过多。	通过调用其他方法获得部分参数，将部分参数替换为可以提供相关信息的对象，或者将部分参数封装为新的参数对象
发散式变化（Divergent Change）	一个类经常因为多种不同的原因在不同的方向上发生变化。	将类中归属不同关注点的部分抽取成不同的类。
霰弹式修改（Shotgun Surgery）	为了实现一个变化（如需求变更）需要在许多类中进行修改。	将相关的属性和方法从不同的类中抽取到同一个已有的类中或者创建一个新的类。
依恋情结（Feature Envy）	一个类中的方法对于其他类中的属性和方法的依赖高过自己所处的类。	将这个方法移动到与之关系密切的类中。如果只是这个方法中的一部分存在这种情况，那么可以先将这部分抽取出来再移动。
数据泥团（Data Clumps）	多个数据项作为类属性或者方法参数在多个地方一起出现。	将这些数据项封装成一个新的类，在它们原来出现的地方加入对这个新类的访问。

定义了22种常见的代码坏味道

《重构：改善既有代码的设计》Martin Fowler.（Fowler，2010）

理解代码坏味道

- 许多代码坏味道都是由于没有很好地遵循面向对象设计思想和原则而造成的，特别是高内聚、低耦合的模块独立性原则
- 一些坏味道之间存在联系，需要加以区分
  - ✓ 发散式变化是指同一个类由于不同的原因而发生变化
  - ✓ 霰弹式修改则是指同一个变化导致不同的类都要进行修改
- 需要注意，每种代码坏味道的评判尺度都可能包含一些主观因素
- 每种代码坏味道对应的重构建议本身也包含着多种因素的权衡，需要根据实际情况进行综合决策

软件重构

- 强调在不改变外部行为的情况下对内部设计进行改进
- 确保软件满足用户需求并确保可用性始终是第一位的，只有在不破坏这一前提的情况下对内部设计的改进才能得到接受和认可
- 通过测试构建软件重构的“防护网”
  - ✓ 要求我们为每个类准备自动化测试用例，并在软件重构过程中频繁运行这些测试，一旦测试不通过马上查找原因并解决然后再继续进行重构
  - ✓ 自动化测试以一种客观可验证的方式给予我们重构的信心
  - ✓ 同时要求以“小步前进”的方式进行重构，即每次只执行一小部分重构然后马上通过测试来进行确认

发现重构机会

- 识别重构机会
  - ✓ 一般可以通过典型的代码坏味道来识别重构机会
  - ✓ 感觉当前的实现方式为软件的进一步维护和演化制造了障碍，而且存在一些已知的参考设计方案可以解决问题时
- 对于进一步维护和演化的障碍经常与所谓的“技术债”（Technical Debt）相关
  - ✓ 软件实现中为了实现某种短期目标而做出的临时性的技术决策（相当于欠下了“债务”），这种权宜之计导致后续软件修改需要付出额外的时间和成本（相当于“利息”）
  - ✓ 许多代码坏味道都是这种技术债的具体体现，例如在已有的条件分支语句中不断增加新的条件分支、拷贝代码修改后用于另一个地方、在代码中“写死”一些变量和逻辑
- 需要善于识别代码中对于进一步维护和演化构成障碍的技术债，并考虑通过重构来消除

常用的软件重构操作

类型	含义	典型重构操作
重新组织方法	调整方法的内容和关系	抽取方法（Extract Method）、内联方法（Inline Method）、内联临时变量（Inline Temp）、以查询代替临时变量（Replace Temp with Query）
在对象之间移动特性	调整类的职责和类间关系	移动字段/方法（Move Field/Method）、抽取类（Extract Class）、内联类（Inline Class）、移除中间人（Remove Middle Man）
重新组织数据	调整数据的封装和访问方式	自封装字段（Self Encapsulate Field）、以对象代替数据值（Replace Data Value with Object）、以字段代替子类（Replace Subclass with Fields）
简化条件表达式	调整和优化代码中的条件表达式	分解条件表达式（Decompose Conditional）、移除控制标记（Remove Control Flag）、以多态代替条件表达式（Replace Conditional with Polymorphism）
简化方法调用	调整和优化对于方法的调用方式	重命名方法（Rename Method）、将查询与修改方法相分离（Separate Query from Modifier）、以工厂方法取代构造方法（Replace Constructor with Factory Method）
处理继承关系	调整和优化类之间的继承关系体系及相关的职责分布	上移字段/方法（Pull Up Field/Method）、下移字段/方法（Push Down Field/Method）、抽取子类/超类（Extract Subclass/Superclass）、抽取接口（Extract Interface）、以委托代替继承（Replace Inheritance with Delegation）
大型重构	在一定范围内开展大规模的软件重构	梳理并分解继承体系（Tease Apart Inheritance）、将过程化设计转化为对象设计（Convert Procedural Design to Objects）、抽取继承体系（Extract Hierarchy）

《重构：改善既有代码的设计》Martin Fowler.（Fowler，2010）

IDE提供的自动化重构操作

- 安全删除（Safe Delete）：在删除文件、类、方法、变量等内容时搜索并显示代码中使用这些待删除元素的情况，供开发人员确认是否需要进行调整
- 复制/移动（Copy/Move）：在不同目录或包之间复制子目录、包、文件或类，在不同路径之间移动包或类或者在不同类之间移动成员属性或方法
- 抽取方法（Extract Method）：将一个方法中的代码片段抽取并移动到一个新的方法中，在原来方法中对应的代码替换为对新方法的调用
- 抽取常量（Extract Constant）：将代码中的硬编码变量（如字符串）抽取为常量
- 抽取字段（Extract Field）：将一个重复出现的变量抽取为类的属性字段
- 抽取参数（Extract Parameter）：将方法中的变量抽取为方法的参数
- 抽取/引入变量（Extract/Introduce Variable）：将方法中不容易理解或者重复出现的表达抽取为变量
- 重命名（Rename）：对一个模块、目录、包、文件、类、方法或各种参数、变量进行重命名，并自动更新所有对它们的引用
- 内联（Inline）：可以视为抽取重构的逆操作，其作用是将一个超类、匿名类、方法、参数、变量在它们被使用的地方展开
- 修改签名（Change Signature）：修改一个类或方法的签名

IntelliJ IDEA（2021.2版） 提供了的自动化重构操作

自动化重构示例

```java
public static void selectSort(int[] arr){
    for(int i=0; i<arr.length; i++){
        int min = i;
        for(int j=i+1; j<arr.length; j++)
            if(arr[j]<arr[min])
                min = j;
        if(i!=min){
            int temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }
    }
}
```

选择待抽取的代码片段
（执行数组元素交换的一段代码）

```java
public static void selectSort(int[] arr){
    for(int i=0; i<arr.length; i++){
        int min = i;
        for(int j=i+1; j<arr.length; j++)
            if(arr[j]<arr[min])
                min = j;
        if(i!=min){
            swap (arr, i, min);
        }
    }
}
```

对新抽取的方法进行重命名

```java
private static void swap(int[] arr, int i, int min) {
    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
```

```java
public static void selectSort(int[] arr){
    for(int i=0; i<arr.length; i++){
        int min = i;
        for(int j=i+1; j<arr.length; j++)
            if(arr[j]<arr[min])
                min = j;
        if(i!=min){
            extracted (arr, i, min);
        }
    }
}
```

完成方法抽取
（原位置自动替换成方法调用）

```java
private static void extracted(int[] arr, int i, int min) {
    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
```

开发人员选择需要重构的代码元素或片段并确定相关的重构选项，IntelliJ IDEA将自动完成相应的重构操作

通常需要组合多种基本重构操作来实现完整的重构目标

本章小结-1

- 软件设计覆盖体系结构设计、组件级详细设计等多个不同层次，扮演着软件需求与实现代码之间的桥梁角色
- 培养软件设计能力首先需要深刻理解与软件设计相关的一些思想，如分解与抽象、关注点分离、模块化、信息隐藏、重构等
- 本章主要围绕面向对象设计方法介绍组件级详细设计
  - ✓ 确定类、接口及其之间的关系，各个类之间应当相对独立
  - ✓ 面向对象设计的一些基本原则可以提供具体的设计指导
  - ✓ 围绕面向对象软件设计存在一些得到广泛应用的通用设计模式，特别是Gamma等人所总结的23种常用的设计模式

本章小结-2

- 横切关注点与面向切面的编程
  - ✓ 面向对象软件设计以类为基本单位的模块化分解无法很好地应对横切关注点的问题
  - ✓ 面向切面的编程方法（AOP）支持横切关注点解耦和模块化封装，为进一步改进软件设计提供了支持
- 按照契约式设计的思想应当为不同类之间的接口定义严格的契约
- 演化式设计
  - ✓ 在软件开发实践中，随着软件演化过程不断调整和完善软件设计方案的演化式设计一般具有更强的现实性和生命力
  - ✓ 这种演化式的软件设计过程往往伴随着对于各种软件设计问题（例如代码坏味道）的分析以及相应的软件设计重构

现代软件工程基础

End

